

Nahradte soubor title-pages.pdf
s Vašemi titulními stránkami
vygenerovanými ze STAGu.

Replace the title-pages.pdf
file with your own title pages
generated from STAG.

Digitální transformace procesů náboru a adaptace pracovníků ve zdravotnické organizaci

ABSTRAKT

V této diplomové práci analyzuji procesy náboru a adaptace pracovníků ve zdravotnické organizaci Krajská zdravotní, a.s., identifikuji jejich klíčová omezení a navrhuji digitální řešení pro jejich sjednocení a zefektivnění. Na základě analytických zjištění, rešerše existujících produktů a návrhu cílové architektury popisuji realizační i provozní aspekty systému včetně distribuované vrstvy inteligentního zpracování dat. Výsledkem je prakticky použitelný návrh, který respektuje multi-tenantní organizační model, požadavky na bezpečnost a auditovatelnost a on-premise provozní podmínky. Součástí práce je také vyhodnocení dalšího směřování vývoje s důrazem na stabilizaci provozu, měřitelnost a postupnou evoluci platformy.

Klíčová slova: Digitalizace, Optimalizace procesů, Softwarová architektura, Adaptační proces, Řízení lidských zdrojů

Digital transformation of recruitment and onboarding processes in a healthcare organization

ABSTRACT

In this diploma thesis, I analyze recruitment and employee onboarding processes in the Czech healthcare organization Krajská zdravotní, a.s., identify their key limitations, and propose a digital solution to unify and optimize them. Based on process analysis, market research of existing products, and target architecture design, I describe implementation and deployment aspects of the system, including a distributed intelligent data processing layer. The result is a practically applicable concept that respects a multi-tenant organizational model, security and auditability requirements, and on-premise operational constraints. The thesis also provides a roadmap for further development focused on operational stabilization, observability, and gradual platform evolution.

Keywords: Digitalization, Process Optimization, Software Architecture, Onboarding Process, Human Resource Management

PODĚKOVÁNÍ

Děkuji vedoucí své práce Ing. Jana Vitvarová Ph. D. za poskytnutou podporu a trpělivost. Mé díky také patří všem testerům za jejich zpětnou vazbu.

Bc. Pavel Vácha

OBSAH

Seznam tabulek	8
Seznam obrázků	10
Seznam zkratk	12
1 Úvod	13
2 Analýza současného stavu	14
2.1 Představení organizace	14
2.1.1 Organizační struktura z pohledu řízení lidských zdrojů	14
2.1.2 Specifika řízení lidských zdrojů ve zdravotnických organizacích	15
2.2 Současný stav procesů náboru pracovníků	16
2.2.1 Proces inzerce volných pozic	17
2.2.2 Proces příjmu a výběru potenciálních kandidátů	18
2.2.3 Proces náboru kandidáta	20
2.2.4 Proces zajištění vstupní agendy	21
2.2.5 Proces adaptace nových zaměstnanců	22
2.3 Identifikace problémů a úzkých míst	23
2.4 Požadavky na digitalizaci procesů	25
2.5 Specifikace funkcionálních a nefunkcionálních požadavků	26
2.5.1 Funkcionální požadavky	26
2.5.2 Nefunkcionální požadavky	29
3 Existující softwarová řešení	30
3.1 Metodika rešerše	30
3.2 Specializované ATS platformy	31
3.2.1 Teamio (Alma Career)	31
3.2.2 Recruitis	31
3.2.3 Datacruit	32
3.2.4 Sloneek (ATS modul)	32
3.3 Onboardingové platformy	32
3.3.1 Onbee	32
3.4 Integrované HCM suite (enterprise)	33
3.4.1 SAP SuccessFactors	33
3.4.2 Workday	33
3.4.3 Oracle Fusion Cloud HCM	33
3.5 Porovnání řešení vůči požadavkům KZ	34

3.6	Identifikovaná omezení dostupných produktů a volba cílového přístupu	35
4	Návrh softwarové architektury	37
4.1	Východiska návrhu a volba architektury	37
4.2	Celkový obraz řešení	39
4.3	Struktura backendu	42
4.4	Datový model a integrační toky	44
4.5	Rámec bezpečnosti a spolehlivosti	47
5	Implementace systému	49
5.1	Struktura řešení	49
5.2	Implementace backendu	51
5.2.1	Použité vzory a techniky v backendu	52
5.2.2	Implementace doménové vrstvy	54
5.2.3	Vynucení architektonických pravidel	54
5.2.4	Implementace spolehlivé asynchronní komunikace	54
5.2.5	Implementace auditní vrstvy	56
5.3	Implementace vrstvy inteligentního zpracování dat	56
5.4	Implementace datové vrstvy a migrací	58
5.4.1	Fyzický datový model	59
5.4.2	Migrace schématu	60
5.4.3	Perzistence dokumentů a infrastrukturní tabulky	60
5.4.4	Implementace bezpečnosti a identity	61
5.5	Implementace onboardingového portálu	62
5.6	Implementace kariérního portálu	62
5.6.1	E2E testování portálu	64
5.6.2	Integrace analytického nástroje Umami	64
5.7	Implementace dohledové vrstvy	65
5.8	Komunikace s národními registry	65
6	Nasazení systému do cílového prostředí	67
6.1	Kontejnerizační model a síťová segmentace	67
6.2	Vydání a nasazení	69
6.3	Správa konfigurace a bezpečnost provozu	70
6.4	Dohledová vrstva	71
6.5	Provozní omezení a mitigace	72
7	Uživatelské testování a zpětná vazba	74
7.1	Cíl a rámec pilotního ověření	74
7.2	Metodický postup a výběr respondentů	74
7.3	Testovací scénáře a hodnotící kritéria	75
7.4	Hlavní zjištění a implikace pro další rozvoj	77
8	Návrh dalšího směřování vývoje	79
8.1	Prioritizační rámec rozvoje	79

8.2	Krátkodobá fáze (0 - 6 měsíců)	79
8.3	Střednědobá fáze (6 - 18 měsíců)	80
8.4	Dlouhodobá fáze (18+ měsíců)	80
8.5	Řízení změny a organizační předpoklady	81
8.6	Závěrečné doporučení kapitoly	81
9	Závěr	82
	Použitá literatura	83

SEZNAM TABULEK

2.1 Klasifikace pozic v KZ	15
2.2 Proces P01 - Vystavení inzerátu	18
2.3 Proces P02 - Příjem a výběr kandidátů k oslovení	19
2.4 Proces P03 - Pohovor a uzavření pracovního poměru	20
2.5 Proces P04 - Nástup zaměstnance	22
2.6 Proces P05 - Adaptace zaměstnance	23
2.7 Kvalitativní hodnocení závažnosti identifikovaných problémů	24
2.8 Požadavky na digitalizaci a jejich vztah na identifikované problémy	25
2.9 Funkcionální požadavky — Kariérní portál	26
2.10 Funkcionální požadavky — Interní řízení nábory	27
2.11 Funkcionální požadavky — Integrace a ověřování kvalifikací	27
2.12 Funkcionální požadavky — Vstupní agenda a adaptace	28
2.13 Funkcionální požadavky — Reporting a multi-tenantní správa	28
2.14 Nefunkcionální požadavky na systém	29
3.1 Hodnoticí rámec řešerše dostupných řešení	30
3.2 Porovnání vybraných řešení podle škvalifikačních kritérií	34
3.3 Porovnání vybraných řešení podle doplňkových kritérií K6-K8	34
4.1 Porovnání variant celkové kompozice řešení	38

5.1 Mapování prvků hexagonální architektury na implementační vrstvy backendu	52
5.2 Typy outbox událostí implementovaných v backendu	55
5.3 Globální role a jejich praktický význam v ReBAC modelu backendu	61
6.1 Hlavní služby v nasazení	68
6.2 Role komponent dohledové vrstvy	72
7.1 Zastoupení rolí v pilotním uživatelském ověření	75
7.2 Sada testovacích scénářů	76
7.3 Ukazatele použité v pilotním uživatelském ověření	77
7.4 Šablona souhrnných výsledků pilotního uživatelského ověření	78

SEZNAM OBRÁZKŮ

2.1 Diagram BPMN znázorňující proces	18
od žádosti po vystavení inzerátu	
2.2 Diagram BPMN znázorňující proces	19
od přijmutí přihlášky po pozvánku na pohovor	
2.3 Diagram BPMN znázorňující proces	21
od pohovoru po uzavření pracovního poměru	
2.4 Diagram BPMN znázorňující proces	22
od nového pracovního poměru po pří- pravu zaměstnance k výkonu práce	
2.5 Diagram BPMN znázorňující proces	23
od připraveného zaměstnance až po adaptovaného zaměstnance	
4.1 Systém v kontextu rolí a externích	40
služeb	
4.2 Logická kompozice řešení	40
4.3 Obecný princip hexagonální architek-	43
tury	
4.4 Návrh hranic backendu v hexagonál-	43
ním uspořádání	
4.5 Zjednodušený diagram hlavních entit	46
a vztahů	
4.6 Tok dat v monitorovací vrstvě od apli-	48
kace po vizualizaci	
5.1 Nasazované komponenty řešení a je-	50
jich hlavní komunikační vazby	
5.2 Realizační tok Outbox-RabbitMQ a vr-	58
stvy inteligentního zpracování dat v implementaci	
5.3 Katalog volných pozic s filtračním pa-	63
nelem a výsledkovou kartou	

6.1 Obecný tok vydání založený na obra-.....	70
zech s distribucí do registru a nasa-	
zením na cílový host	

SEZNAM ZKRATEK

KZ	Krajská zdravotní, a.s.
HR	Human Resources
LZ	Lékaři a farmaceuti
NL- ZP	Nelékařští zdravotničtí pracovníci
IT	Informační technologie
ÚZIS	Ústav zdravotnických informací a statistiky ČR
BO- ZP	Bezpečnost a ochrana zdraví při práci
THP	technickohospodářských pracovníků
ATS	Applicant Tracking System
HCM	Human Capital Management
SSO	Single Sign-On
NRZP	Národní registr zdravotnických pracovníků

1 ÚVOD

Digitalizace personálních procesů ve zdravotnictví není pouhým převodem formulářů do elektronické podoby. V prostředí velké zdravotnické organizace se v jednom procesu střetává vysoký objem náboru, legislativní požadavky na odbornou způsobilost, rozdílné potřeby jednotlivých pracovišť i tlak na rychlé obsazování pozic. Pokud jsou tyto okolnosti dlouhodobě řešeny převážně e-mailem, sdílenými tabulkami a lokálními zvyklostmi, nevzniká jen administrativní zátěž. Vzniká také nepřehlednost, která zpomaluje nábor, komplikuje nástup nových zaměstnanců a oslabuje schopnost řídit proces na úrovni celé organizace.

Téma této práce proto vychází z praktické potřeby Krajské zdravotní, a.s. sjednotit a zpřehlednit nábor a adaptaci pracovníků napříč odštěpnými závody. Nejde přitom jen o zefektivnění jednotlivých dílčích kroků, ale o vytvoření takového řešení, které propojí inzerci, práci s uchazeči, vstupní agendu i adaptaci nového zaměstnance do jednoho srozumitelného a provozně udržitelného celku.

Cílem práce je analyzovat současný stav těchto procesů, identifikovat jejich hlavní nedostatky a navrhnout řešení jejich digitalizace formou odpovídající softwarové architektury. Součástí práce je i porovnání existujících produktů a zdůvodnění, proč v podmínkách Krajské zdravotní, a.s. dává smysl vlastní vývoj. Na analytickou a rešeršní část navazuje návrh architektury, popis implementace, nasazení do cílového prostředí a pilotní uživatelské ověření řešení. Práce se soustředí na proces od vzniku personální potřeby až po adaptaci nového zaměstnance.

2 ANALÝZA SOUČASNÉHO STAVU

Efektivní digitalizaci nelze stavět na domněnkách o tom, jak organizace funguje, ale na přesném porozumění skutečnému průběhu procesů, jejich aktérům a slabým místům. V souladu s metodikou procesního řízení dle Řepy [1] proto analytická část nejprve mapuje výchozí stav a teprve na tomto základě formuluje požadavky na budoucí informační systém.

Analýza současného stavu procesů náboru a adaptace pracovníků v Krajské zdravotní, a.s. proto nejprve ukotvuje organizační a doménová specifika zdravotnického prostředí. Následně zachycuje klíčové procesy pomocí notace BPMN (Business Process Model and Notation), identifikuje jejich úzká místa a převádí zjištěné nedostatky do podoby funkčních a nefunkčních požadavků.

2.1 PŘEDSTAVENÍ ORGANIZACE

Krajská zdravotní, a.s. (KZ) je akciová společnost vlastněná Ústeckým krajem a současně největší poskytovatel lůžkové i ambulantní zdravotní péče v regionu. Vznikla v roce 2007 sloučením pěti nemocnic do jediné právní entity. Smyslem tohoto kroku nebyla pouze organizační konsolidace, ale i sjednocení řízení, nákupu a sdílení odborných kapacit napříč regionem.

V roce 2021 se struktura KZ dále rozšířila. Do společnosti byla integrována **Nemocnice Litoměřice, o.z.**, a následně bylo převzato i zdravotnické zařízení v Rumburku, které dnes funguje jako **KZ - Masarykova nemocnice v Ústí nad Labem, o.z., detašované pracoviště Rumburk**. Tím se počet odštěpných závodů zvýšil na sedm.

S více než jedenácti tisíci zaměstnanci patří KZ mezi největší zaměstnavatele v Ústeckém kraji. Pro tuto práci je podstatné zejména to, že značnou část personálu tvoří zdravotničtí pracovníci s regulovanou odbornou způsobilostí. Nábor a adaptace zde proto nejsou jen administrativní agendou, ale procesem, který musí současně splnit provozní, legislativní i odborné požadavky.

2.1.1 ORGANIZAČNÍ STRUKTURA Z POHLEDU ŘÍZENÍ LIDSKÝCH ZDROJŮ

KZ je řízená centrálně, avšak jednotlivé nemocnice mají zároveň prostor samostatně řešit běžné provozní a personální záležitosti. Každý odštěpný závod má vlastní personální zázemí Human Resources (HR) pro operativní agendu, správu

smluv, docházku i řízení adaptace, ale současně se řídí jednotnou metodikou a strategickými cíli holdingu.

Z pohledu digitalizace je důležité, že systém nesmí uvažovat organizaci jako jeden homogenní celek. Musí respektovat samostatnost jednotlivých závodů a zároveň umožnit centrální dohled, reporting a správu pravidel. V architektonické terminologii jde o požadavek na *multi-tenantní* řešení, ve kterém každý odštěpný závod vystupuje jako samostatný tenant sdílející společnou infrastrukturu i metodické řízení.

2.1.2 SPECIFIKA ŘÍZENÍ LIDSKÝCH ZDROJŮ VE ZDRAVOTNICKÝCH ORGANIZACÍCH

Řízení lidských zdrojů v prostředí KZ se od běžného korporátního modelu liší především tím, že velká část pracovních rolí podléhá regulované odborné způsobilosti. Náborový systém proto nemůže řešit pouze evidenci kandidátů a komunikaci s nimi. Musí současně hlídat kvalifikace, zákonné podmínky výkonu povolání a návaznost těchto informací na adaptační proces.

Rozdílné nároky se projevují už v samotné kategorizaci pracovníků. Zaměstnanci KZ spadají do několika skupin, z nichž každá vyžaduje jiný rozsah dokladů, jiný způsob ověření způsobilosti a často i odlišný průběh adaptace. Pro další text zavádím zejména kategorie Lékaři a farmaceuti (LZ), Nelékařští zdravotníci (NLZP) a Informační technologie (IT), které se v procesech opakují.

Tabulka 2. 1: Klasifikace pozic v KZ

Kategorie (dle KS)	Typické pozice v KZ	Klíčová specifika pro HR
LZ	Atestovaní lékaři, lékaři v přípravě, farmaceuti	Sledování specializačního vzdělávání (atestace), IP-VZ, evidence v ČLK.
NLZP	Všeobecné sestry, dětské sestry, porodní asistentky	Odborná a specializovaná způsobilost, vzdělávání pod NCONZO.
Ostatní NLZP a JOP	Radiologičtí asistenti, fyzioterapeuti, sanitáři, kliničtí psychologové	Certifikované kurzy, akreditované stáže, specifické nároky na praxi.
THP a Provoz	Ekonomové, IT specialisté, údržba, stravovací provoz	Zařazení dle Katalogu prací, standardní zákoník práce.

Hlavním specifikem je právě práce v rámci **regulované odborné způsobilosti**. Základním pilířem HR procesů v KZ je soulad s legislativním rámcem pro výkon

zdravotnických povolání. Nábor a následná správa zaměstnanců se zde opírají zejména o dvě normy:

- Zákon č. 95/2004 Sb. - upravuje získávání odborné a specializované způsobilosti u lékařů, zubních lékařů a farmaceutů. Systém musí sledovat průběh předatestační přípravy, platnost členství v ČLK/ČSK a zařazení do specializačních oborů.
- Zákon č. 96/2004 Sb. - definuje podmínky pro NLZP. Zde je kritické sledování odborné způsobilosti a schopnosti vykonávat povolání bez odborného dohledu (v souladu s aktuální metodikou Ministerstva zdravotnictví a Ústav zdravotnických informací a statistiky ČR (ÚZIS)).

Informační systém proto musí umět validovat doklady o dosaženém vzdělání, zaznamenat výsledek jejich kontroly a následně pracovat i s informacemi o registracích v profesních komorách nebo o odborné způsobilosti pod dohledem. Bez této vrstvy by digitalizace sice urychlila administrativu, ale neřešila by to, co je v prostředí zdravotnické organizace skutečně kritické.

Dalším specifikem je **kontinuální nábor**. Na rozdíl od prostředí, kde se nábor spouští jen při výjimečné potřebě, musí zdravotnická organizace dlouhodobě reagovat na fluktuaci, demografický vývoj i celorepublikový nedostatek pracovníků, zejména mimo hlavní centra. [2] Systém proto musí být připraven na opakované a souběžné zpracování velkého množství pozic i uchazečů.

Podpisem pracovní smlouvy končí práce náboráře a personálního oddělení. Následuje **vícetupňový adaptační proces**, který zahrnuje nejen standardní nástupní agendu a školení Bezpečnost a ochrana zdraví při práci (BOZP), ale i odborné zapracování, práci s interní dokumentací, seznámení s nemocničními systémy a ověření připravenosti na konkrétním pracovišti. Právě vazba mezi nábořem, vstupní agendou a adaptací je proto v této práci klíčová.

2.2 SOUČASNÝ STAV PROCESŮ NÁBORU PRACOVNÍKŮ

Před zahájením digitalizace stojí správa uchazečů v KZ převážně na „svaté dvojici“ sdílených tabulek a e-mailové komunikace, kterou doplňují lokální zvyklosti jednotlivých závodů. Takové řešení může fungovat v menší organizaci, ale v podmínkách zdravotnického holdingu začíná postupně brzdit celý proces. Nábor zde pak nepůsobí jako jeden plynulý celek, ale jako řada navazujících administrativních kroků, mezi nimiž se informace ztrácejí nebo zbytečně přepisují.

Pro účely analýzy jsem proto mapoval skutečný průběh procesů pomocí strukturovaných rozhovorů s vedoucím personálního oddělení a s náboráři jednotlivých odštěpných závodů. Vycházel jsem nejen z formální dokumentace, ale i z popisu

reálné praxe, tedy z toho, jak proces skutečně probíhá při běžném provozu, včetně neformálních dohod a obcházení chybějících nástrojů.

Pro zajištění terminologické konzistence jsou v této kapitole používány následující pojmy:

- **Uchazeč** o zaměstnání je fyzická osoba, která reaguje na konkrétní zveřejněnou pracovní pozici a doručí zaměstnavateli svou přihlášku (životopis, motivační dopis nebo jinou formu reakce).
- **Kandidát** je uchazeč, který prošel prvotním roztříděním a byl vybrán do další fáze výběrového řízení (např. k osobnímu pohovoru nebo odbornému posouzení).
- **Zájemce** o zaměstnání je osoba, která projeví zájem o zaměstnání v KZ bez vazby na konkrétní vyhlášenou pracovní pozici (tzv. talent pool).

2.2.1 PROCES INZERCE VOLNÝCH POZIC

Inzerce volných pracovních pozic je v současném stavu rozložena mezi několik nezávislých kanálů:

- **Webové portály třetích stran** - Jobs.cz, Práce.cz (provozovatel LMC), portál MPSV
- **Nemocniční nástěnky** - fyzické nástěnky v areálech nemocnic
- **Webové stránky nemocnic** - statické stránky s omezenou aktualizací
- **Sociální síť** — LinkedIn

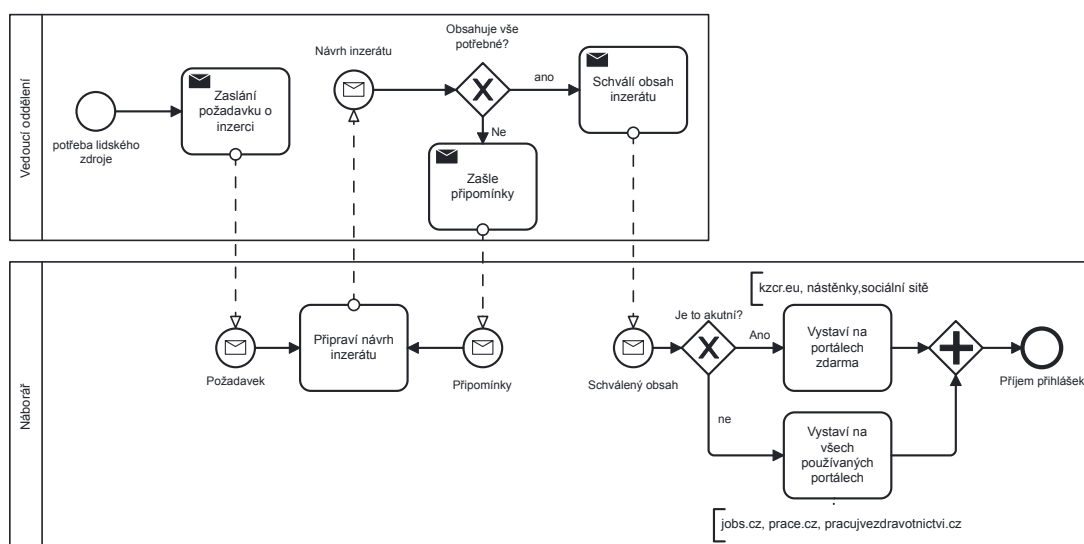
Absence vlastního kariérního portálu znamená, že uchazeč nemá jednotný vstupní bod, ve kterém by našel aktuální nabídky všech závodů, základní informace o zaměstnavateli i možnost okamžitě reagovat na pozici. Už na této úrovni tedy vzniká roztříštěnost, kterou organizace dále přenáší i do interní práce s inzeráty.

Proces začíná tím, že vedoucí oddělení identifikuje potřebu obsadit pozici a předává ji personálnímu oddělení. Vstupní informace však nejsou standardizované. Rozsah i kvalita zadání se liší podle konkrétního vedoucího, takže HR často nejprve doplňuje chybějící údaje a až poté připravuje text inzerátu. Samotné schvalování proto probíhá v několika e-mailových iteracích a místo plynulého procesu vzniká komunikační smyčka.

Po schválení textu následuje ruční publikace na jednotlivých platformách. Každý portál má vlastní formulář, vlastní strukturu dat i vlastní způsob následné aktualizace. Změna termínu nebo úprava požadavků proto neznamena jednu opravu v centrálním systému, ale opakování stejné práce na více místech. Důsledkem je nízká transparentnost, vyšší časová náročnost a slabý přehled vedoucích pracovníků o tom, kde se jejich pozice právě nachází a jaké reakce na ni přišly.

Tabulka 2. 2: Proces P01 - Vystavení inzerátu

Identifikátor procesu:	P01
Název procesu:	Vytvoření inzerátu pro pracovní pozici
Zákazník:	Vedoucí daného oddělení v odštěpném závodě
Vlastník procesu:	Náborář, vedoucí daného oddělení v odštěpném závodě
Účel:	Vytvoření inzerátu na poptávanou pozici
Produkt:	Inzerát
Technické prostředky:	E-mail, webové stránky KZ, webové portály třetích stran, nemocniční nástěnky, sociální sítě
Metrika:	Rychlost od žádosti po vystavení inzerátu
Nedostatky:	Manuální publikace inzerátu, již neaktuální inzeráty na portálech, nejednotý přehled o vydaných financích, dlouhé administrativní kolečko mezi vedoucím a náborářem



Obrázek 2. 1: Diagram BPMN znázorňující proces od žádosti po vystavení inzerátu

2.2.2 PROCES PŘÍJMU A VÝBĚRU POTENCIÁLNÍCH KANDIDÁTŮ

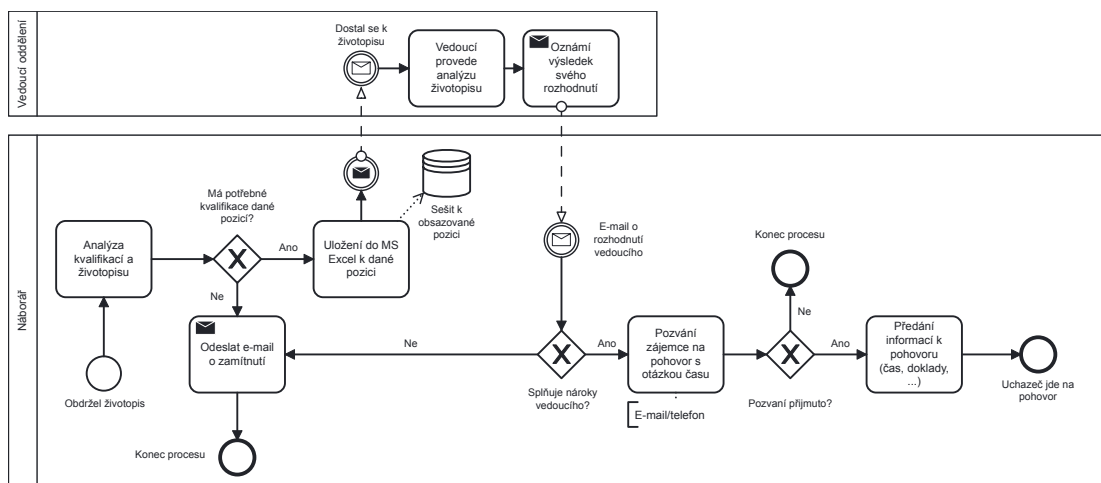
Přihlášky dnes přicházejí z různých zdrojů a náborář je musí ručně sjednocovat do tabulek a lokálně uložených příloh. Samotné získání přehledu o kandidátech je tak oddělenou administrativní činností, nikoli přirozenou součástí náborového systému. Výsledkem je nekonzistence dat mezi pracovišti a nízká dohledatelnost dokumentů.

Prvotní třídění uchazečů probíhá převážně manuálně. U zdravotnických pozic to znamená vyhledávat v každém životopise údaje o vzdělání, odborné způsobilosti a další podmínky, které rozhodují o dalším postupu. Bez automatických filtrů a strukturovaných dat je tento krok pomalý a zároveň náchylný k přehlédnutí důležitých informací. Užší výběr se následně předává vedoucím oddělení znovu e-mailem, takže rozhodovací proces zůstává roztříštěný.

Kritickým nedostatkem je i absence systematické databáze uchazečů a zájemců. Informace o kandidátech, kteří nebyli přijati nebo reagovali bez vazby na konkrétní pozici, se dále nevyužívají. Organizace tím přichází o možnost vracet se k relevantním kontaktům při dalším náboru a zvyšuje si závislost na opakované inzerci, která s sebou nese finanční zátěž.

Tabulka 2. 3: Proces P02 - Příjem a výběr kandidátů k oslovení

Identifikátor procesu:	P02
Název procesu:	Příjem přihlášek a výběr kandidátů k oslovení
Zákazník:	Vedoucí daného oddělení v odštěpném závodě
Vlastník procesu:	Náborář, vedoucí daného oddělení v odštěpném závodě
Účel:	Výběr kandidátů, jenž splňují požadavky obsazované pozice
Produkt:	Kandidát pozvaný na pohovor
Technické prostředky:	E-mail, telefon, tabulkový procesor, Webex
Metrika:	Rychlost od vystavení inzerátu po oslovení kandidáta
Nedostatky:	Neexistence centrální databáze uchazečů o pozici, neexistence databáze potenciálních zájemců o zaměstnání v KZ



Obrázek 2. 2: Diagram BPMN znázorňující proces od přijetí přihlášky po pozvání na pohovor

2.2.3 PROCES NÁBORU KANDIDÁTA

Proces náboru pracovníka (P03) navazuje na okamžik, kdy je kandidát pozván na pohovor. V této fázi už nejde pouze o výběr vhodného člověka, ale o splnění všech podmínek nutných pro vznik pracovněprávního vztahu. Prakticky se zde propojuje rozhodnutí vedoucího oddělení s administrativními a zákonnými kroky personálního oddělení.

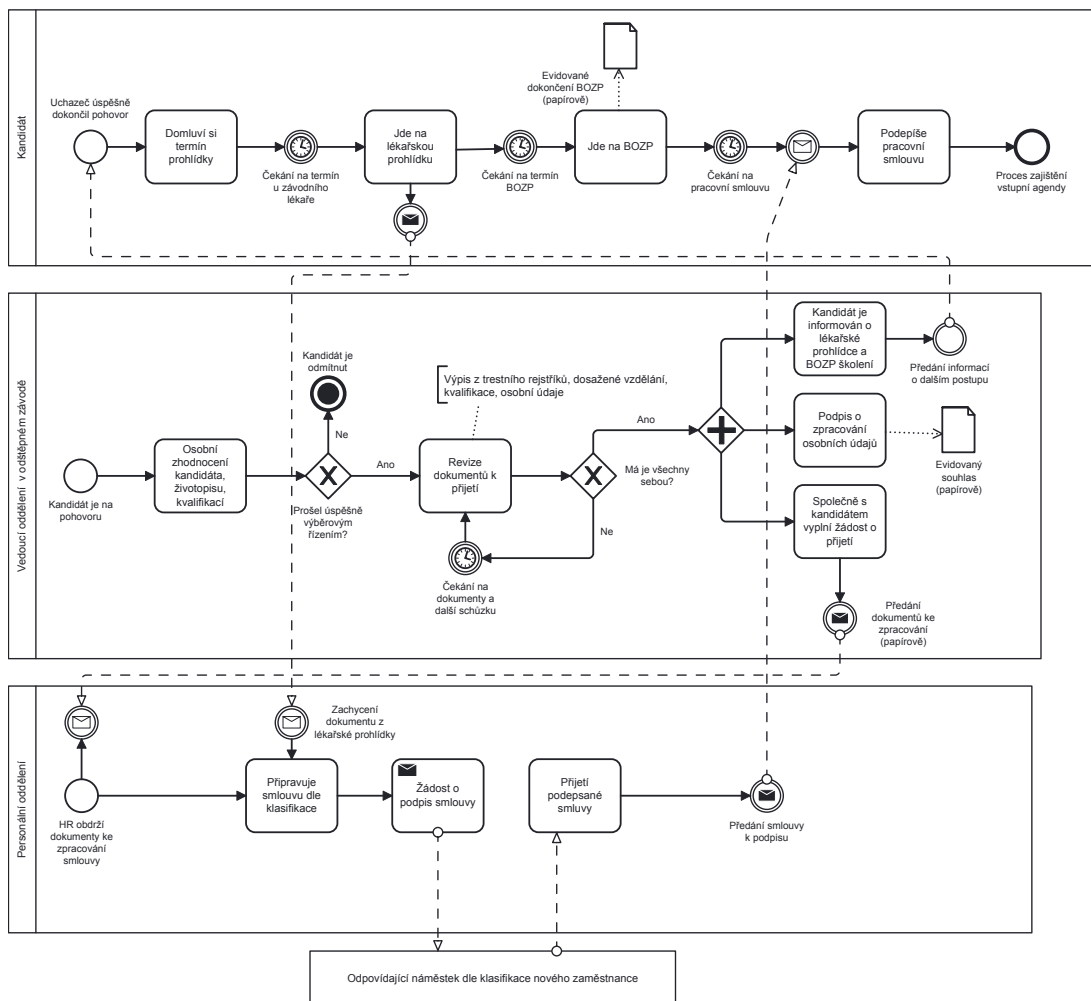
Po úspěšném pohovoru začíná sběr identifikačních údajů a povinných dokumentů. Kandidát dokládá výpis z rejstříku trestů, doklady o vzdělání, odborné kvalifikaci a další podklady podle charakteru pozice. Následují vstupní lékařská prohlídka a školení BOZP. Každý z těchto kroků je nezbytný, ale v současném stavu probíhá převážně manuálně a bez jednotného přehledu o tom, co již bylo dodáno a co ještě chybí.

Proces tak sice formálně vede k uzavření pracovní smlouvy, ale z provozního hlediska v něm chybí centrální koordinace. Informace jsou rozloženy mezi e-maily, listinné podklady a jednotlivé pracovníky, což prodlužuje průchod celou fází a zvyšuje riziko administrativního zdržení.

Tabulka 2. 4: Proces P03 - Pohovor a uzavření pracovního poměru

Identifikátor procesu:	P03
Název procesu:	Pohovor a uzavření pracovního poměru
Zákazník:	Vedoucí oddělení, PAM
Vlastník procesu:	Vedoucí oddělení, personální a mzdové oddělení
Účel:	Odborné posouzení kandidáta a splnění podmínek pro vznik pracovněprávního vztahu
Produkt:	Podepsaná pracovní smlouva
Technické prostředky:	Listinné dokumenty, e-mail, telefon
Metrika:	Doba od pohovoru po podpis pracovní smlouvy
Nedostatky:	Manuální sběr dokladů, vícestupňová administrativa, papírová komunikace

Celý proces je znázorněn na diagramu níže (viz obr. Obrázek 2. 3), který zachycuje jednotlivé kroky od pohovoru až po podpis pracovní smlouvy. Diagram ukazuje návaznost činností mezi kandidátem, vedoucím oddělení a personálním oddělením. Zároveň je z obrázků hned patrná roztříštěnost procesu, zejména v oblasti předávání dokumentů a koordinace jednotlivých kroků.



Obrázek 2. 3: Diagram BPMN znázorňující proces od pohovoru po uzavření pracovního poměru

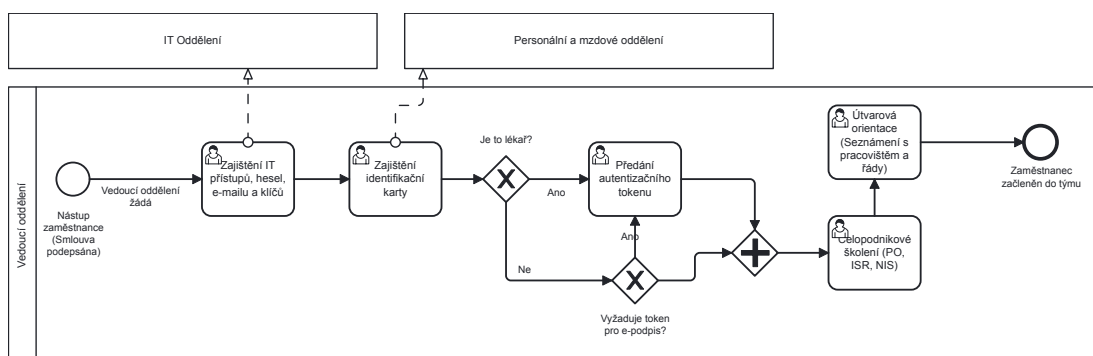
2.2.4 PROCES ZAJIŠTĚNÍ VSTUPNÍ AGENDY

V okamžiku nástupu zaměstnance začíná fáze, ve které je třeba připravit člověka na skutečný výkon práce. Patří sem vydání identifikační karty, nastavení docházky, přístupových oprávnění a v některých případech i prostředků pro elektronické podepisování. Z pohledu procesu je důležité, že tyto kroky nevykonává jediná role, ale více útvarů s rozdílnou odpovědností.

Vedle technických přístupů probíhá i útvarová orientace nového zaměstnance. Vedoucí pracovník jej seznamuje s pracovištěm, provozním řádem, dokumentací a očekáváním spojeným s výkonem práce. Problém současného stavu nespočívá v tom, že by tyto kroky neexistovaly, ale v tom, že nejsou vedeny v jednotném přehledu. Organizace proto obtížně zjišťuje, co bylo splněno, co se zdrželo a kde se nový zaměstnanec může zaseknout ještě před plným nástupem do své nové role.

Tabulka 2. 5: Proces P04 - Nástup zaměstnance

Identifikátor procesu:	P04
Název procesu:	Nástup zaměstnance a zajištění přístupových prostředků
Zákazník:	Nový zaměstnanec
Vlastník procesu:	PAM, IT, vedoucí oddělení
Účel:	Zajištění připravenosti zaměstnance k výkonu práce
Produkt:	Zaměstnanec vybaven ID kartou, přístupy a případně tokenem
Technické prostředky:	ID karta, docházkový systém, autentizační token, interní IT systémy
Metrika:	Doba od podpisu smlouvy po plnou funkčnost zaměstnance
Nedostatky:	Oddělené nastavování oprávnění, ruční aktivace přístupů, absence jednotného postupu



Obrázek 2. 4: Diagram BPMN znázorňující proces od nového pracovního poměru po přípravu zaměstnance k výkonu práce

2.2.5 PROCES ADAPTACE NOVÝCH ZAMĚSTNANCŮ

Adaptační proces tvoří most mezi formálním nástupem a plnohodnotným výkonem práce. V prostředí KZ je tato fáze zvláště důležitá u zdravotnických pozic, protože nestačí pouze administrativně dokončit nástup. Je třeba zajistit i odborné zapracování v podmínkách konkrétního pracoviště.

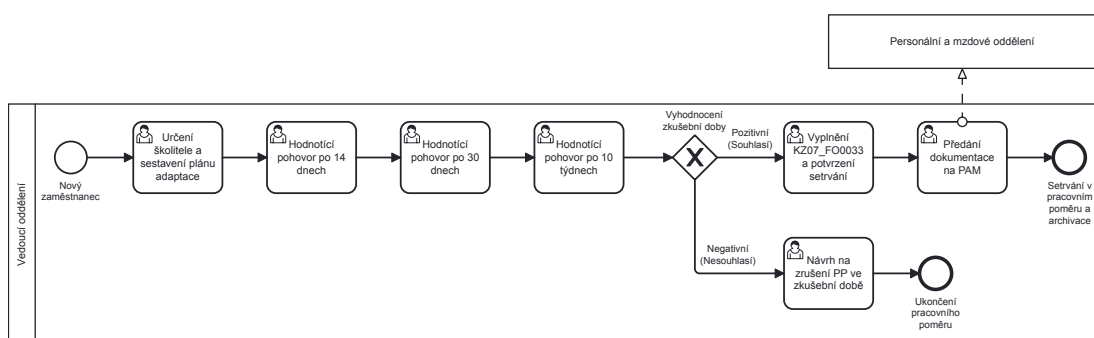
Obecná část adaptace se vztahuje na všechny zaměstnance a probíhá zejména ve zkušební době. Vedoucí zaměstnanec určuje školitele, nastavuje adaptační plán a průběžně hodnotí jeho plnění. Odborná část je pak výraznější u zdravotnických profesí, kde se sleduje nejen zvládnutí organizačních pravidel, ale i osvojení pracovních postupů, standardů a očekávané míry samostatnosti.

V současném stavu je však adaptace z velké části vedena papírově. To komplikuje průběžný monitoring, znesnadňuje auditní dohled a prakticky znemožňuje

centrálně vyhodnocovat, jak adaptace probíhá napříč odštěpnými závody. Organizace tak sice adaptaci vykonává, ale nemá k dispozici jednotný digitální obraz o jejím stavu a výsledcích.

Tabulka 2. 6: Proces P05 - Adaptace zaměstnance

Identifikátor procesu:	P05
Název procesu:	Adaptační proces zaměstnance
Zákazník:	Vedoucí oddělení, zaměstnanec
Vlastník procesu:	Vedoucí zaměstnanec, školitel
Účel:	Začlenění zaměstnance do pracovního a odborného prostředí
Produkt:	Plně adaptovaný zaměstnanec
Technické prostředky:	Adaptační formuláře, hodnotící pohovory
Metrika:	Míra setrvání po zkušební době, fluktuace
Nedostatky:	Papírová dokumentace, chybějící monitoring, slabý reporting



Obrázek 2. 5: Diagram BPMN znázorňující proces od připraveného zaměstnance až po adaptovaného zaměstnance

2.3 IDENTIFIKACE PROBLÉMŮ A ÚZKÝCH MÍST

Na základě analýzy současného stavu procesů nábory a adaptace v KZ, konzultací s HR pracovníky jednotlivých nemocnic a pozorování reálného průběhu procesů jsem identifikoval následující klíčové problémy. Problémy jsou kategorizovány podle oblastí dopadu a doplněny o kvalitativní hodnocení závažnosti.

Identifikované problémy mají kumulativní charakter a v provozní praxi se vzájemně zesilují. Významným akceleračním faktorem je vysoká personální dynamika organizace, která dosahuje průměrně 110 nástupů měsíčně (v sezónních špičkách až 180). Typická měsíční struktura nástupů přitom zahrnuje přibližně 10 pracovníků kategorie LZ, 50 NLZP a 50 technickohospodářských pracovníků

(THP)/dělnických profesí. V takovém objemu se manuální administrativa stává kritickým omezením propustnosti celého procesu.

Z analytického pohledu lze problémy rozdělit do tří tematických oblastí. První oblast představuje **datová fragmentace** (P1-P3), kdy jsou informace rozptýleny mezi e-mailovou komunikací, lokální soubory a tabulkové přehledy jednotlivých závodů. Druhou oblast tvoří **procesní a řídicí nedostatky** (P4-P9), zejména chybějící auditní stopa, nízká standardizace rozhodování a slabá transparentnost stavu náboru i vstupní agendy. Třetí oblast je **digitálně nepodpořená adaptace** (P10), která omezuje možnost systematicky řídit zapracování nových zaměstnanců a vyhodnocovat jeho úspěšnost.

Tabulka 2. 7: Kvalitativní hodnocení závažnosti identifikovaných problémů

ID	Problém	Závažnost	Hlavní dopad
P1	Tabulková a papírová agenda	Vysoká	Riziko ztráty dat, neefektivní práce
P2	Chybějící evidence uchazečů	Vysoká	Ztráta kandidátů, nemožnost analytiky
P3	Neexistující evidence zájemců	Střední	Ztráta potenciálních kandidátů
P4	Absence auditní stopy	Vysoká	Právní rizika, GDPR, neschopnost auditu
P5	Omezený reporting	Střední	Rozhodování bez datové opory
P6	Komunikační smyčka	Vysoká	Prodloužení doby obsazení pozice
P7	Manuální publikace	Střední	Časová náročnost, nekonzistence
P8	Nestrukturované hodnocení	Střední	Subjektivní výběr
P9	Nepřehledný stav vstupní agendy	Vysoká	Problémy při nástupu zaměstnance
P10	Papírová adaptace	Vysoká	Nemožnost monitoringu a reportingu

Souhrnně lze konstatovat, že stávající procesní model již neodpovídá rozsahu organizace ani požadavkům na datově řízené rozhodování. Zjištění této kapitoly proto představují přímý vstup pro definici požadavků na cílové řešení.

2.4 POŽADAVKY NA DIGITALIZACI PROCESŮ

Na základě provedené analýzy procesů a identifikovaných problémů (Kapitola 2.3) lze formulovat klíčové požadavky na digitalizaci procesů náboru a adaptace. Každý požadavek je odůvodněn vazbou na identifikované problémy.

Požadavky R1-R6 představují minimální funkční rámec cílového systému. Každý požadavek je formulován tak, aby byl implementovatelný, ověřitelný při testování a současně jednoznačně navázaný na problémy identifikované v předchozí sekci.

Tabulka 2. 8: Požadavky na digitalizaci a jejich vazba na identifikované problémy

ID	Požadavek	Vymezení a cíl	Vazba
R1	Multi-tenantní architektura	Respektovat strukturu KZ (samostatné závody) a umožnit centrální řízení a reporting.	P1, P2, P5
R2	Veřejný kariérní portál	Poskytnout jednotný vstupní bod s aktuálními nabídkami, online přihláškou a registrací zájemce (tzv. talent pool).	P2, P3, P6, P7
R3	Interní administrační rozhraní	Centralizovat správu inzerce, kandidátů, výběrových kroků, kontrolu kvalifikací i přípravu vstupní agendy.	P1, P2, P4, P5, P6, P7, P8, P9
R4	Adaptační portál	Digitalizovat adaptační plány, průběžné hodnocení, notifikace termínů a souhrnný přehled stavu adaptace.	P1, P4, P5, P10
R5	Integrace s NRZP	Automatizovat ověřování odborné způsobilosti zdravotnických pracovníků přes napojení na NRZP.	P1, P4
R6	Reporting a analytika	Zajistit dashboardy a export metrik náboru a adaptace pro závody i centrální úroveň.	P5

Tabulka výše ukazuje, že jednotlivé požadavky nepokrývají izolované části procesu, ale řeší problémy napříč celým životním cyklem kandidáta. Současně je patrná snaha o propojení náboru a adaptace do jednoho integrovaného systému, který umožní lepší přehled, efektivnější koordinaci a kvalitnější rozhodování na úrovni jednotlivých závodů i centrály.

2.5 SPECIFIKACE FUNKCIONÁLNÍCH A NEFUNKCIONÁLNÍCH POŽADAVKŮ

Na základě formulovaných požadavků na digitalizaci (R1-R6) jsem v této sekci provedl jejich dekompozici na konkrétní funkcionální a nefunkcionální požadavky, které slouží jako vstup pro návrh softwarové architektury a implementaci systému.

2.5.1 FUNKCIONÁLNÍ POŽADAVKY

Funkcionální požadavky definují konkrétní chování systému, tedy co systém musí umožňovat svým uživatelům nebo jakých výstupů musí být schopen. Požadavky jsou kategorizovány podle oblastí systému a prioritizovány metodou MoSCoW (Must have, Should have, Could have, Won't have).

Tabulka 2. 9 shrnuje funkcionální požadavky na veřejnou část systému, tedy kariérní portál určený pro uchazeče. Požadavky se zaměřují na vytvoření jednoho a plně digitálního vstupního bodu pro uchazeče. Cílem je eliminace ostatních komunikačních kanálů, zejména e-mailu, telefonické a papírové komunikace, které v současném stavu vedou k roztržitosti dat a zvýšené administrativní zátěži.

Tabulka 2. 9: Funkcionální požadavky — Kariérní portál

ID	Požadavek	Priorita	Vazba
F01	Systém zobrazí veřejný seznam aktuálních pracovních nabídek ze všech odštěpných závodů KZ s možností filtrování podle závodu, kategorie pozice, typu úvazku a lokality	Must	R2
F02	Uchazeč se může přihlásit na vybranou pozici prostřednictvím online formuláře s přiložením životopisu a dalších dokumentů	Must	R2
F03	Systém umožní registraci zájemce o zaměstnání v KZ i bez vazby na konkrétní pozici (talent pool)	Must	R2, R3
F04	Kariérní portál prezentuje informace o zaměstnavatelských benefitech, stipendijních programech a pracovním prostředí v KZ	Should	R2
F05	Detail pracovní nabídky obsahuje strukturovaný popis pozice, požadavky na kvalifikaci, nabízené podmínky a kontaktní informace	Must	R2
F06	Systém umožní uchazeči kontaktovat organizaci prostřednictvím kontaktního formuláře	Should	R2

Tabulka 2. 10 shrnuje funkcionální požadavky na interní část systému určenou pro náboráře, personální a mzdové oddělení a vedoucí oddělení. Požadavky pokrývají celý průběh náboru od založení požadavku na obsazení pozice až po výběr kandidáta.

Tabulka 2. 10: Funkcionální požadavky — Interní řízení náboru

ID	Požadavek	Priorita	Vazba
F07	Systém umožní založení a správu požadavku na obsazení pozice včetně schvalovacího procesu mezi vedoucím oddělení a HR	Must	R3
F08	Systém bude vést centrální evidenci kandidátů s historií změn stavů napříč celým náborovým procesem	Must	R3
F09	Systém umožní plánování pohovorů, přiřazení odpovědných osob a evidenci výsledků jednotlivých kol	Must	R3
F10	Systém poskytne standardizované komunikační šablony (pozvánka, zamítnutí, žádost o doplnění podkladů) a jejich evidenci	Should	R3
F11	Systém umožní strukturované hodnocení kandidátů dle předem definovaných kritérií	Should	R3, R6
F12	Vedoucí oddělení bude mít online přehled o stavu svých náborových požadavků bez nutnosti ad-hoc e-mailových dotazů	Must	R3, R6

Tabulka 2. 11 shrnuje funkcionální požadavky na integraci systému s externími zdroji. Součástí je také auditní stopa v souladu s principy řízené správy dat.

Tabulka 2. 11: Funkcionální požadavky — Integrace a ověřování kvalifikací

ID	Požadavek	Priorita	Vazba
F13	Systém umožní automatizované ověření odborné způsobilosti pracovníků napojením na národní registr zdravotnických pracovníků	Should	R5
F14	Systém upozorní náboráře, pokud uchazeč nemá platný záznam v NRZP nebo má omezenou způsobilost	Should	R5
F15	Systém eviduje výsledky ověření kvalifikace včetně časového razítka a zdroje	Must	R3, R5

Tabulka 2. 12 shrnuje funkcionální požadavky na digitalizaci vstupní agendy a adaptačního procesu. Požadavky reagují na manuální řízení těchto činností zavedením strukturovaných seznamů, evidence plnění a upozornění.

Tabulka 2. 12: Funkcionální požadavky — Vstupní agenda a adaptace

ID	Požadavek	Priorita	Vazba
F16	Systém vytvoří vstupní checklist nástupu podle typu pozice a organizační jednotky	Must	R3, R4
F17	Systém umožní evidovat plnění přednástupních povinností (BOZP, vstupní prohlídka, smluvní dokumentace, identifikační karta, přístupová oprávnění)	Must	R3, R4
F18	Systém bude automaticky upozorňovat odpovědné role na blížící se termíny nebo prodlení u vstupní agendy	Should	R4
F19	Vedoucí nebo školitel bude moci založit adaptační plán obsahující úkoly, termíny a hodnoticí milníky	Must	R4
F20	Systém umožní průběžné hodnocení plnění adaptačního plánu a evidenci hodnoticích rozhovorů	Must	R4, R6
F21	Systém poskytne přehled stavu adaptací podle závodu, oddělení a typu pracovní pozice	Should	R4, R6

Tabulka 2. 13 shrnuje požadavky na oddělení dat a centrální pohled na nábor.

Tabulka 2. 13: Funkcionální požadavky — Reporting a multi-tenantní správa

ID	Požadavek	Priorita	Vazba
F22	Systém podporuje multi-tenantní model, kde každý odštěpný závod je samostatným tenantem s vlastními daty, uživateli a konfigurací	Must	R1
F23	Uživatelé s rolí centrálního administrátora mají přístup k datům a reportům napříč všemi tenanty	Must	R1, R6
F24	Systém poskytuje přehled s klíčovými metrikami (počet otevřených pozic, průměrná doba obsazení, poměr přihlášek/přijetí, stav adaptací) na úrovni závodu i celé organizace	Should	R6
F25	Systém umožní export reportů do formátu PDF a CSV	Could	R6

2.5.2 NEFUNKCIONÁLNÍ POŽADAVKY

Nefunkcionální požadavky definují kvalitativní vlastnosti systému, které nejsou přímo pozorovatelné jako konkrétní funkce, ale podstatně ovlivňují použitelnost, spolehlivost a udržitelnost systému.

Tabulka 2. 14: Nefunkcionální požadavky na systém

ID	Oblast	Požadavek
NF01	Bezpečnost	Systém musí zajistit ochranu osobních údajů uchazečů a zaměstnanců v souladu s GDPR (nařízení 2016/679) a zákonem č. 110/2019 Sb. o zpracování osobních údajů
NF02	Bezpečnost	Autentizace uživatelů musí být zajištěna prostřednictvím protokolu OAuth 2.0 s integrací do existujícího SSO poskytovatele organizace
NF03	Bezpečnost	Systém musí podporovat řízení přístupů podle rolí alespoň ve stupních: administrátor, HR pracovník a vedoucí oddělení
NF04	Dostupnost	Systém musí dosahovat dostupnosti alespoň 99,5 % v pracovních dnech v čase 6:00-22:00
NF05	Výkon	Odezva běžných operací nesmí v 95. percentilu překročit 2 sekundy.
NF06	Přístupnost	Kariérní portál musí být responzivní a navržený v souladu se zásadami WCAG 2.1 na úrovni AA
NF07	Nasaditelnost	Systém musí být nasaditelný na infrastruktuře organizace (on-premise) prostřednictvím kontejnerizace (Docker)
NF08	Udržitelnost	Zdrojový kód musí být verzován v systému pro správu verzí (Git) a dokumentován v rozsahu umožňujícím předání jinému vývojovému týmu
NF09	Lokalizace	Veškerá uživatelská rozhraní musí být v českém jazyce, systém musí správně pracovat s českou diakritikou ve všech vrstvách (databáze, API, UI)
NF10	Kompatibilita	Kariérní portál musí být kompatibilní s aktuálními verzemi prohlížečů Chrome, Firefox, Safari a Edge
NF11	Auditovatelnost	Systém musí uchovávat auditní záznamy o změnách klíčových entit (pozice, kandidát, adaptace) minimálně po dobu 5 let

3 EXISTUJÍCÍ SOFTWAREVÁ ŘEŠENÍ

Rozhodnutí o vlastním vývoji je obhajitelné pouze tehdy, pokud je zřejmé, proč dostupná řešení nedokážou naplnit požadavky organizace bez nepřiměřených kompromisů. Tato kapitola proto porovnává vybraná řešení hodnotí jejich vhodnost vzhledem k podmínkám KZ.

3.1 METODIKA REŠERŠE

Rešerše byla provedena jako komparativní posouzení řešení ze tří kategorií. Applicant Tracking System (ATS), onboardingové platformy (vstupní agenda a adaptace) a Human Capital Management (HCM) systémy. Na základě analyzovaných požadavků vznikla kritéria pro hodnocení uvedené níže v Tabulka 3. 1.

Tabulka 3. 1: Hodnoticí rámec rešerše dostupných řešení

ID	Hodnoticí kritérium	Vazba na požadavky	Typ
K1	Pokrytí celého náborového procesu	R2, R3	Must
K2	Pokrytí vstupní agendy a adaptace zaměstnance	R4	Must
K3	Podpora holdingového členění (oddělená data + centrální dohled)	R1	Must
K4	Integrace a rozšiřitelnost (API, datové vazby)	R5	Must
K5	Možnost provozu na vlastní infrastruktuře	NF07	Must
K6	Reporting a analytika napříč závody	R6	Should
K7	Lokalizace pro české prostředí	NF09	Should
K8	Podpora publikace inzerce na externích portálech	R2	Could

Hodnocení vychází z veřejně dostupné dokumentace výrobců, produktových stránek a oficiálních ceníků (**stav k 21. 11. 2025**).

Pro účely rozhodnutí byla kritéria typu **Must** chápána jako diskvalifikační. Pokud řešení nesplní kterékoli kritérium K1-K5, není vhodné jako cílová platforma pro digitalizaci procesů KZ, i když může být přínosné jako dílčí integrační komponenta.

3.2 SPECIALIZOVANÉ ATS PLATFORMY

Specializované ATS nástroje jsou zaměřeny především na počáteční fázi náboru. Silné jsou zejména v oblasti správy pracovních pozic, evidence uchazečů, komunikace s nimi a zveřejňování pracovních nabídek. Pro účely této práce však není klíčové pouze to, jak efektivně dokážou nábor zahájit, ale zda na něj dokážou plynule navázat i v oblasti vstupní agendy a adaptačního procesu. Právě tyto fáze jsou v prostředí zdravotnické organizace výrazně složitější než v běžném komerčním náboru.

3.2.1 TEAMIO (ALMA CAREER)

Teamio lze považovat za referenční ATS řešení českého trhu, zejména díky jeho úzkému propojení s dominantními inzertními kanály Jobs.cz a Práce.cz. Tato vazba je v podmínkách KZ významná, protože umožňuje snížit transakční náklady spojené s publikací inzerce a konsolidací reakcí uchazečů. Oficiální dokumentace současně uvádí možnost automatického importu pozic z interních systémů i z portálů provozovaných Alma Career. [3]

Z hlediska funkčního pokrytí Teamio naplňuje klíčové ATS scénáře a v komerční nabídce deklaruje i preboarding/onboarding rozšíření. [4] Z pohledu požadavků této práce je však zásadní provozní model produktu, který je koncipován jako služba bez lokální instalace. [5] Tento model je sice vhodný pro rychlé uvedení do provozu, avšak nevytváří přímý soulad s požadavkem KZ na on-premise nasazení a interní kontrolu datového provozu.

Z analytického hlediska je proto Teamio vhodné vnímat spíše jako specializovanou integrační komponentu pro inzerci a sběr kandidátských reakcí než jako plnohodnotný cílový systém pro end-to-end řízení náboru, vstupní agendy a adaptace.

3.2.2 RECRUITIS

Recruitis je ATS platforma orientovaná na digitalizaci náborového workflow, práci s talent poolem a reportingovou podporu náborových aktivit. Veřejné materiály zároveň uvádějí vazbu na onboardingové řešení Onbee, což potvrzuje interope-

rabilitu systému, ale zároveň ukazuje, že náborová a adaptační vrstva jsou řešeny odděleně. [6]

Ve vztahu k enterprise požadavkům je důležitá deklarovaná podpora REST API integrace, Single Sign-On (SSO) a více právních subjektů. [6] Tyto parametry zvyšují použitelnost v heterogenním organizačním prostředí. Přesto i zde platí, že řešení komplexního adaptačního procesu by muselo být zajištěno navazujícím externím modulem, což zvyšuje závislost na mezisystémové koordinaci.

3.2.3 DATACRUIT

Datacruit deklaruje ATS funkcionalitu doplněnou o automatizační prvky a integrační otevřenost. [7] Z hlediska této práce je klíčové, že oficiální ceník u enterprise varianty uvádí možnost implementace na vlastní infrastrukturu zákazníka a licenční model, který není čistě subscription-based. [8]

Tento parametr činí Datacruit z porovnávaných ATS systémů nejbližším kandidátem ve vztahu k požadavku NF07. Funkční těžiště produktu je však nadále v oblasti náboru. Plnohodnotná podpora adaptačního procesu zdravotnických pracovníků by tedy vyžadovala další produktovou vrstvu nebo zakázkové rozšíření, což zvyšuje architektonickou i provozní složitost cílového řešení.

3.2.4 SLONEEK (ATS MODUL)

Sloneek nabízí ATS modul jako součást širší HR platformy. [9] Tento přístup je výhodný z pohledu sjednocení základních personálních agend, avšak dostupná dokumentace současně uvádí omezení v oblasti přímé integrace ATS modulu na externí platformy. [9]

V prostředí KZ tak Sloneek představuje využitelnou variantu pro obecnou digitalizaci HR činností, nikoli však jednoznačně vhodného kandidáta pro cílové řešení s požadovanou mírou specializace na zdravotnické procesy, multi-tenantní řízení a on-premise provoz.

3.3 ONBOARDINGOVÉ PLATFORMY

3.3.1 ONBEE

Onbee představuje specializovanou onboarding/offboarding platformu zaměřenou na procesní koordinaci nástupu zaměstnance, práci s checklisty a správu související dokumentace. Ve veřejné produktové dokumentaci jsou deklarovány integrační možnosti na ATS/HRIS, podpora SSO i možnost on-premise instalace. [10]

Z hlediska hodnoticího rámce tak Onbee velmi dobře pokrývá část požadavků spojených s adaptací (R4) a částečně i provozní požadavky NF07. Jeho limit

spočívá v tom, že nejde o plnohodnotné ATS řešení, takže by organizace musela paralelně provozovat a integračně udržovat další náborový systém. Vznikla by tak vícevrstvá architektura s vyšší závislostí na kvalitě integračních vazeb.

3.4 INTEGROVANÉ HCM SUITE (ENTERPRISE)

Integrované HCM platformy sledují jiný strategický cíl než specializované ATS nebo onboardingové nástroje. Snaží se pokrýt co největší část zaměstnaneckého cyklu v jednom datovém a procesním modelu. Z pohledu enterprise architektury jde o logický přístup, protože snižuje fragmentaci dat. V praxi však bývá vykoupěn vyšší implementační náročností, větším rozsahem změnového řízení a velmi často i cloudovým provozním modelem.

3.4.1 SAP SUCCESSFACTORS

SAP SuccessFactors je výrobcem prezentován jako cloud-based HCM suite s moduly pro recruiting i onboarding. [11, 12] Z hlediska funkční šíře jde o robustní platformu schopnou pokrýt velkou část HR procesů, včetně centralizovaného reportingu a governance.

V podmínkách KZ je však hlavním limitem provozní model orientovaný na cloud a nutnost rozsáhlé konfigurace na lokální zdravotnické specifikum. Rizikem je i vyšší implementační setrvačnost při změnách procesního modelu během projektu.

3.4.2 WORKDAY

Workday deklaruje jednotnou cloud platformu pro HR a související procesy včetně talent acquisition. [13, 14] Ve srovnání s tradičním modulárním přístupem poskytuje silný důraz na datovou konzistenci a analytiku napříč procesy.

Stejně jako u SAP SuccessFactors je však v této práci rozhodující nesoulad mezi cloudovým provozním modelem a požadavkem KZ na provoz na vlastní infrastrukturu. Dalším faktorem je náročnost lokalizace na české regulační prostředí a specifika zdravotnických rolí.

3.4.3 ORACLE FUSION CLOUD HCM

Oracle Fusion Cloud HCM je výrobcem prezentován jako kompletní cloudové řešení zahrnující recruiting i onboarding procesy. [15, 16] Z hlediska procesního pokrytí jde o srovnatelně široký koncept jako u předchozích enterprise platform.

V případě KZ však i zde převažuje stejná bariéra: nesoulad s požadavkem na on-premise provoz a vysoká míra přizpůsobení nutná pro české zdravotnické procesy, zejména ve vazbě na národní registry a interní organizační členění holdingu.

3.5 POROVNÁNÍ ŘEŠENÍ VŮČI POŽADAVKŮM KZ

Následující tabulky shrnují vybraná řešení podle diskvalifikačních kritérií K1-K5 a doplňkových kritérií K6-K8. Hodnocení je provedeno kvalitativně ve škále **Ano / Částečně / Ne** na základě veřejně doložitelných informací.

Pro Tabulka 3. 2 platí: K1 = nábor, K2 = adaptace, K3 = multi-tenantní podpora, K4 = integrace/API, K5 = on-premise provoz.

Tabulka 3. 2: Porovnání vybraných řešení podle diskvalifikačních kritérií

Řešení	K1	K2	K3	K4	K5
Teamio	Ano	Částečně	Částečně	Ano	Ne
Recruitis	Ano	Částečně	Částečně	Ano	Ne
Datacruit	Ano	Částečně	Částečně	Ano	Ano
Sloneek	Částečně	Částečně	Částečně	Částečně	Ne
Onbee	Ne	Ano	Částečně	Ano	Ano
SAP SuccessFactors	Ano	Ano	Ano	Ano	Ne
Workday	Ano	Ano	Ano	Ano	Ne
Oracle HCM	Ano	Ano	Ano	Ano	Ne

Tabulka 3. 3: Porovnání vybraných řešení podle doplňkových kritérií K6-K8

Řešení	K6 Reporting	K7 Lokalizace	K8 Inzerce
Teamio	Ano	Ano	Ano
Recruitis	Ano	Ano	Ano
Datacruit	Ano	Částečně	Ano
Sloneek	Ano	Ano	Částečně
Onbee	Částečně	Ano	Ne
SAP SuccessFactors	Ano	Částečně	Částečně
Workday	Ano	Částečně	Částečně
Oracle HCM	Ano	Částečně	Částečně

Interpretace výsledků v Tabulka 3. 2 ukazuje, že žádný z hodnocených produktů nesplňuje současně všechna diskvalifikační kritéria K1-K5. Enterprise HCM platformy dosahují vysokého funkčního pokrytí náboru i adaptace, avšak jejich cloudová orientace je v přímém rozporu s provozními omezeními KZ. Specializované ATS platformy naopak přinášejí vysokou efektivitu v akviziční části náboru, nicméně navazující adaptační proces obvykle pokrývají jen částečně nebo prostřednictvím odděleného nástroje.

Z porovnání doplňkových kritérií vyplývá, že rozdíly mezi produkty se zmenšují v oblastech reportingu a obecné lokalizace, které jsou v komerčních řešeních obvykle dobře pokryty. Rozhodující je proto schopnost naplnit specifické požadavky domény zdravotnictví a interní architektonická omezení organizace, nikoli pouze šíře standardních HR funkcí.

Specifickým požadavkem zdravotnického prostředí je napojení na ověřování odborné způsobilosti zdravotnických pracovníků (NRZP), které je v českém eGovernment ekosystému dostupné jako samostatná služba. [17] Ve veřejné dokumentaci hodnocených komerčních produktů nebyla k datu rešerše doložena explicitní, hotová podpora tohoto procesu bez doplňkového vývoje nebo integrační nadstavby.

3.6 IDENTIFIKOVANÁ OMEZENÍ DOSTUPNÝCH PRODUKTŮ A VOLBA CÍLOVÉHO PŘÍSTUPU

Zjištění ukazují, že hlavní limit dostupných produktů neleží v absenci jednotlivých funkcí. Leží v nesouladu mezi jejich produktovou logikou a cílovou architekturou KZ. První omezení představuje provozní model. Významná část funkčně robustních enterprise platforem je koncipována jako cloud-native služba, zatímco KZ vyžaduje provoz na vlastní infrastruktuře z důvodu interní governance, bezpečnostních politik a provozní autonomie.

Druhé omezení se týká procesní kontinuity. U specializovaných ATS řešení je často vysoká kvalita náborové části, avšak adaptační proces bývá realizován v odděleném nástroji. Vzniká tak architektura „best of breed“, která je funkčně flexibilní, ale z pohledu provozu přináší rizika roztříštěnosti dat, složitější správu identit a vyšší náklady na dlouhodobé integrační testování.

Třetí omezení spočívá v doménové specifitě českého zdravotnictví. Hodnocené produkty jsou navrženy jako obecně použitelné HR platformy pro více odvětví a zemí. To je jejich komerční výhoda, ale současně limit v situaci, kdy organizace potřebuje cíleně implementovat lokální procesní logiku, například ověřování odborné způsobilosti, auditovatelný průchod zdravotnickou adaptací nebo detailní vazbu na interní předpisy jednotlivých závodů.

Čtvrtým omezením je dlouhodobá provozní závislost na produktových plánech dodavatelů. U klíčových procesů, které jsou pro KZ strategické, představuje tato závislost významné riziko. Organizace by musela adaptovat interní postupy podle vývojového směru třetí strany, což je v rozporu s cílem řídit transformaci procesů primárně podle vlastních potřeb.

Na základě uvedených zjištění jsem zvolil jako nejvhodnější přístup založený na vývoji vlastního řešení. Tento přístup mi umožňuje navrhnout jednotný datový model pro nábor, vstupní agendu i adaptaci, implementovat multi-tenantní architekturu odpovídající holdingovému uspořádání a současně respektovat požadavek na on-premise provoz bez kompromisních výjimek.

Vlastní řešení zároveň snižuje riziko, že kritické doménové požadavky budou odloženy kvůli externí produktovému plánu. Integrace, které jsou pro KZ skutečně přínosné, lze realizovat selektivně a v řízeném rozsahu, například pro distribuci inzerce nebo výměnu dat s externími registry. Tím se kombinuje výhoda procesní suverenity s pragmatickým využitím existujícího softwarového ekosystému.

Z metodického hlediska tedy tato kapitola nevede k závěru, že komerční produkty jsou obecně nevhodné. Vede k závěru, že pro konkrétní kombinaci požadavků R1-R6 a NF01-NF12 je v podmínkách KZ vlastní řešení variantou s nejvyšší mírou strategické a provozní shody.

4 NÁVRH SOFTWAREVÉ ARCHITEKTURY

Architektura navrženého řešení musela odpovědět na praktickou otázku, jak digitalizovat nábor a adaptaci v prostředí, které je současně organizačně členité, regulatorně citlivé a provozně omezené infrastrukturou provozovanou v prostorách organizace (on-premise). Nešlo tedy o návrh obecně elegantního systému, ale o takovou architekturu, která bude obhajitelná vůči požadavkům Krajské zdravotní, a.s. a zároveň zůstane dlouhodobě udržitelná.

4.1 VÝCHODISKA NÁVRHU A VOLBA ARCHITEKTURY

Architektonická rozhodnutí jsem odvozoval přímo z provozního kontextu KZ. Organizace není nově vznikající podnik budovaný od nuly s jedním produktem a jedním týmem. Jde o holding se sedmi odštěpnými závody, regulovanými daty, různorodými uživatelskými rolemi a omezenými provozními kapacitami. Z toho plyne, že architektura musí být nejen technicky správná, ale i srozumitelná pro další rozvoj a bezpečně provozovatelná.

Nejsilnějším strukturálním požadavkem je víceorganizační izolace dat (R1). Každý závod musí pracovat se svými daty, ale centrální vedení potřebuje průřezový pohled. Tento požadavek proto nelze řešit jen na úrovni oprávnění v uživatelském rozhraní. Musí být propsán do datového modelu, aplikačních služeb i do způsobu, jakým se vyhodnocují přístupová pravidla. Vedle toho musí systém pokrýt celý tok od zveřejnění pozice přes nábor a vstupní agendu až po adaptaci zaměstnance (R2–R4), být připraven na napojení externích služeb (R5) a poskytovat data pro řízení procesu (R6).

Z kvalitativních atributů [18] jsem kladl největší důraz na bezpečnost, auditovatelnost a provozní udržitelnost. V prostředí zdravotnictví není auditní stopa doplňkem, ale základní podmínkou důvěryhodnosti systému. Stejně tak lokální provoz znamená, že nemohu spoléhat na komfort spravovaných cloudových služeb. Architektura proto musí minimalizovat zbytečnou distribuovanou složitost a současně připustit řízený růst tam, kde je skutečně potřebný.

Pro volbu cílové architektury bylo nejprve nutné oddělit dvě úrovně rozhodování, které se v praxi často směšují. První se týká vnitřního uspořádání jedné aplikace. Zde je relevantní vrstvené oddělení odpovědností [18], modulární členění podle

doménových celků [19] a hexagonální vymezení hranice mezi jádrem a okolím prostřednictvím portů a adaptérů [20].

Druhá úroveň se týká celkové kompozice řešení, tedy otázky, zda budou jeho části provozovány jako jeden celek, nebo jako více samostatných běhových komponent se všemi důsledky distribuované architektury [21]. Tyto dvě úrovně si nekonkurují. První určuje, jak chránit doménu před infrastrukturou, druhá rozhoduje, kolik distribuované složitosti je účelné přijmout.

Tabulka 4. 1 proto neporovnává vrstvenou, modulární a hexagonální architekturu mezi sebou. Porovnává pouze varianty celkové kompozice řešení. Klasický monolit, modulární monolit, plně mikroslužbový přístup a hybridní model. Smyslem tohoto srovnání je určit, jaký celek je pro podmínky KZ přiměřený z hlediska provozu, integrací a dalšího rozvoje.

Tabulka 4. 1: Porovnání variant celkové kompozice řešení

Varianta	Silné stránky	Rizika
Klasický monolit	Jednoduchý vývoj, nasazení a provoz v počáteční fázi.	Růst vnitřních vazeb a horší dlouhodobá udržitelnost.
Modulární monolit	Provozní jednoduchost při zachování jasnějších doménových hranic.	Nutnost důsledně hlídat hranice modulů a závislosti.
Přístup s mikroslužbami	Nezávislý vývoj, nasazení a škálování jednotlivých částí.	Vysoká distribuovaná složitost a provozní režie.
Hybridní model	Stabilní transakční jádro lze doplnit o samostatné části s odlišnými nároky.	Nutnost řídit integrační kontrakty a provozní hranice mezi komponentami.

Klasický monolit by sice zjednodušil první implementační kroky, ale rychle by rozostřil hranice mezi náborovou logikou, integracemi, auditem a analytickými funkcemi. Plně mikroslužbový přístup by naopak od začátku vnesl síťovou komunikaci, distribuované selhávání a vyšší provozní režii do projektu, který je vyvíjen iterativně a s omezenými kapacitami [21]. Samotný modulární monolit proto vycházel jako nejvhodnější základ pro transakční část řešení, protože udržuje jeden autoritativní stav pro identitu, oprávnění, audit i workflow. Méně vhodný by však byl tam, kde se mění běhový profil, výkonové nároky nebo knihovní závislosti, zejména u inteligentního zpracování životopisů a pracovních pozic.

Jako nejlepší řešení se proto ukázal hybridní model. Na úrovni celku tvoří základ modulární transakční jádro provozované jako jeden hlavní backend. Uvnitř tohoto jádra se současně uplatňuje hexagonální uspořádání a principy doménově řízeného návrhu (Domain-Driven Design, dále jen DDD), protože pomáhají oddělit stabilnější doménu od databáze, vrstvy předávání zpráv a externích služeb. Mimo jádro jsou vyčleněny pouze ty části, které mají odlišný běhový profil nebo jiné integrační nároky, konkrétně komponenty `cv_processor` a `job_processor`.

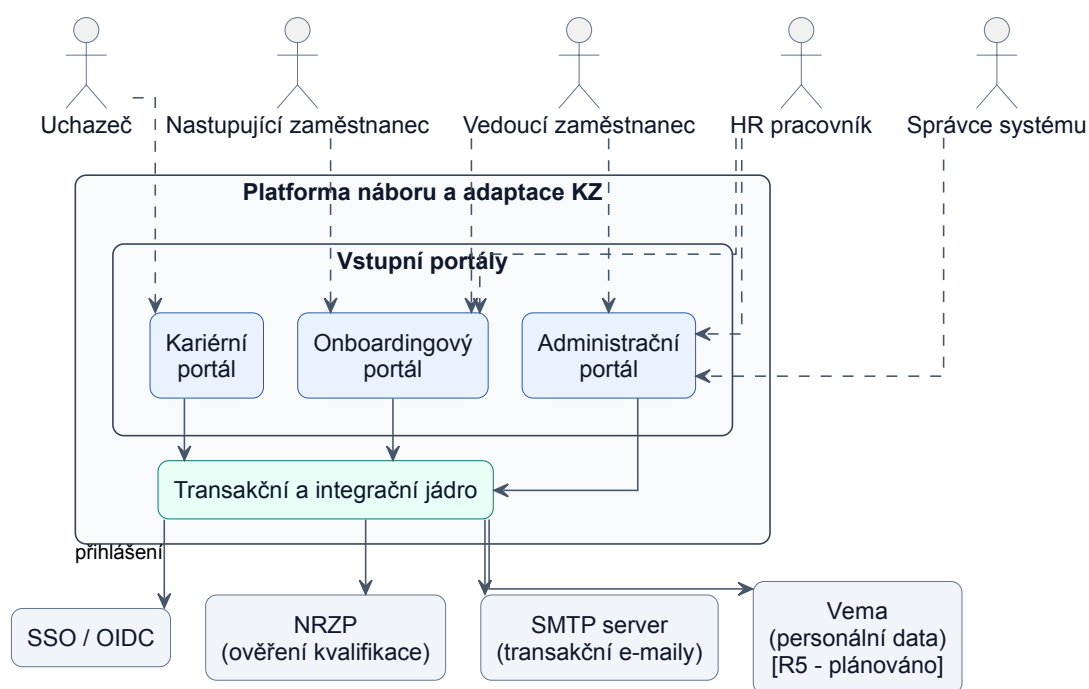
Pro víceorganizační, auditovatelný a lokálně provozovaný systém je tento model vhodný právě proto, že odlišuje různé zdroje složitosti. Hlavní proces nábory a adaptace potřebuje jedno autoritativní jádro, v němž se konzistentně vyhodnocují organizační hranice, identita, oprávnění i auditní stopa. Naopak procesory pro inteligentní zpracování dat a některé integrační okraje mohou být odděleny bez toho, aby se rozpadla transakční konzistence domény. Lokální provoz současně zvyhodňuje omezený počet kritických běhových komponent, protože každý další distribuovaný prvek zvyšuje nároky na dohled, zálohování a řešení incidentů.

Cenou za tento přístup je vyšší návrhová i provozní náročnost než u čistého monolitu. Systém obsahuje více integračních kontraktů, více nasazovacích hranic a vyšší nároky na monitorování. Přínosem je naopak výrazně lepší oddělení domény, integrací a infrastruktury, a tedy i vyšší udržitelnost a schopnost dlouhodobého rozvoje řešení v prostředí, kde nelze předpokládat stabilní soubor požadavků ani jednoduchý provoz.

4.2 CELKOVÝ OBRAZ ŘEŠENÍ

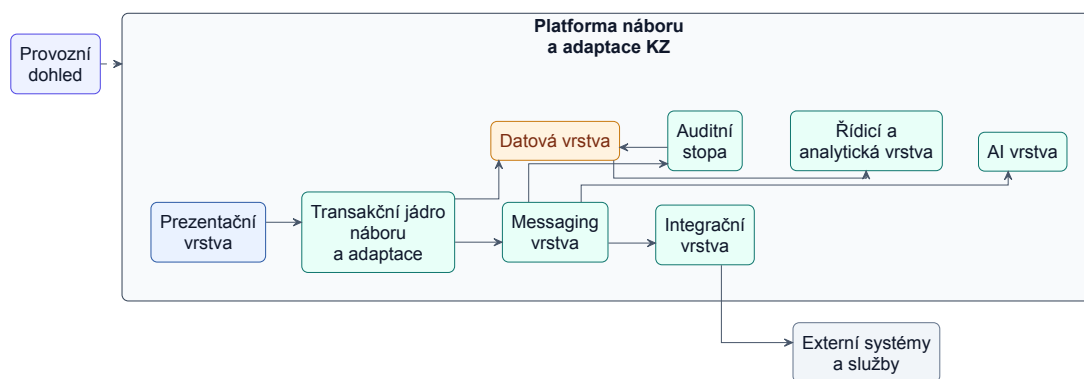
Prvním krokem je zachytit systém v kontextu rolí a vnějších služeb. Tím se vyjasní, proč řešení nevzniklo jako jedna univerzální interní aplikace, ale jako soustava více vstupních bodů nad společným transakčním a integračním jádrem.

Z Obrázek 4. 1 je patrné, že systém obsluhuje několik odlišných skupin uživatelů s různými cíli a různou mírou oprávnění. Uchazeči vstupují přes veřejný kariérní portál, interní náborová agenda je soustředěna do administračního portálu a nástup se sledováním adaptace do portálu pro řízení nástupu nad stejným doménovým jádrem. Na straně vnějších vazeb jsou rozhodující jednotné přihlašování SSO prostřednictvím protokolu OIDC, Národní registr zdravotnických pracovníků (NRZP) a e-mailová infrastruktura. Právě tyto závislosti potvrzují, že řešení musí být navrženo jako integrační uzel, nikoli jen jako izolovaná evidence.



Obrázek 4. 1: Systém v kontextu rolí a externích služeb

Na kontextový pohled navazuje logická kompozice řešení. Jejím smyslem není popsat konkrétní nasazovací topologii ani technologickou skladbu, ale ukázat, z jakých stavebních bloků se systém skládá a proč jsou hranice mezi nimi vedeny právě tímto způsobem.



Obrázek 4. 2: Logická kompozice řešení

Systém musí současně zvládat transakční agendu náboru a adaptace, komunikaci s okolními službami, výpočetně náročné zpracování i průběžné vyhodnocování procesu. Pokud by se tyto odpovědnosti soustředily do jedné vrstvy, docházelo by ke směšování stabilní domény s proměnlivými integračními a analytickými požadavky, což by vedlo ke ztrátě přehlednosti i rozšiřitelnosti.

Na tuto situaci odpovídá Obrázek 4. 2. V centru návrhu stojí transakční jádro náboru a adaptace, které nese doménová pravidla a hlavní procesní tok. K němu

se připojuje prezentační vrstva obsluhující veřejný kariérní portál i interní rozhraní, datová vrstva uchováající stav procesu, samostatná vrstva předávání zpráv (messaging) pro asynchronní přeposílání úloh a událostí, integrační vrstva zajišťující řízený kontakt s okolím, auditní stopa pro dohledatelnost změn a oddělená vrstva s inteligentním zpracováním dat. Diagram záměrně abstrahuje od konkrétních technologií, protože jejich realizační rozpad patří až do implementační kapitoly. Na architektonické úrovni je důležité obhájit logiku hranic, nikoli vyjmenovat každou nasazenou službu.

Návrh vychází přímo z požadavků R1–R6. Požadavky R2–R4 definují tři navazující situace (veřejný vstup, interní řízení, adaptace), což vede k oddělení prezentační vrstvy od jednotného transakčního jádra. Požadavek R1 pak promítá holdingovou strukturu KZ do datové vrstvy, která tak nese nejen stav systému, ale i organizační kontext.

Oddělení řídicí a analytické vrstvy je klíčové. Systém nemá pouze evidovat průběh náboru a adaptace, ale také poskytovat podklady pro jeho řízení a průběžné zlepšování. Proto do této vrstvy patří nejen vyhodnocování průchodnosti náboru a adaptace, ale i analytika kariérního portálu. Ta umožňuje sledovat, jak se potenciální uchazeči na portálu chovají, kde ztrácejí pozornost, ve kterých krocích proces opouštějí a které části nabídky nebo formulářů snižují míru dokončení přihlášky. Právě proto je tato vrstva oddělena od technického provozního dohledu. Provozní dohled odpovídá na otázku, zda systém běží zdravě, zatímco řídicí a analytická vrstva vysvětluje, co se v náborovém procesu skutečně děje.

Samostatné zachycení auditní stopy je přímou odpovědí na problém P4, tedy absenci auditní stopy v původním procesu, i na nefunkcionální požadavek NF11. V logické kompozici ji proto nevedu jako další podnikovou oblast, ale jako podpůrnou architektonickou schopnost systému. Auditní událost vzniká v jádře, je předána přes vrstvu předávání zpráv a následně trvale uložena do datové vrstvy. Tím zůstává audit mimo hlavní cestu požadavků a současně je zachována dohledatelnost změn nad klíčovými entitami.

Stejně podstatná je integrační vrstva. Její role nespočívá jen v napojení na NRZP, které přímo vychází z požadavku R5. Stejně důležitá je schopnost bezpečně připojovat i další okolní systémy a služby, které se promítají do každodenního provozu náboru a adaptace, například identitní služby, VEMA, národní registry nebo komunikační infrastrukturu. Nejde tedy o jednu konkrétní integraci, ale o řízenou hranici mezi transakčním jádrem a cizími systémy. Transakční jádro zde pouze vyjadřuje, jakou schopnost od okolí potřebuje, například vyhledat kvalifikaci, interního uživatele nebo ověřit identitu, zatímco integrační vrstva přebírá odpovědnost za cizí protokol, datový model i chybové stavy vnější služby. Smyslem této vrstvy je převzít integrační složitost na sebe, aby se nepropisovala přímo do transakční logiky.

Vedle ní stojí vrstva předávání zpráv, která neřeší, na jaký vnější systém se systém napojuje, ale jak jsou úlohy a události předávány mimo hlavní transakční tok.

4.3 STRUKTURA BACKENDU

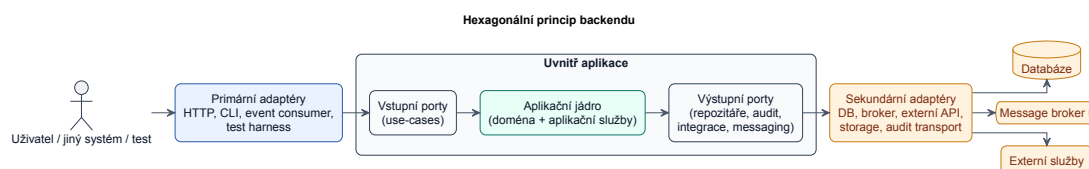
V předchozí části byl systém popsán na úrovni logické kompozice, tedy jako soubor hlavních částí řešení a jejich odpovědností. Tento pohled ukazuje, jak do sebe zapadají veřejné portály, transakční jádro, integrační vrstva, vrstva předávání zpráv, analytická část, auditní stopa a provozní dohled.

Nejcitlivější částí celého řešení je backend, protože právě v něm se setkávají doménová pravidla, datové hranice, bezpečnost i integrace. Jádro backendu proto stavím na hexagonální architektuře, kterou Cockburn popsal jako vzor portů a adaptérů (Ports and Adapters) [20]. Motivací tohoto vzoru je oddělit aplikační logiku od uživatelského rozhraní, databáze a dalších zařízení tak, aby systém bylo možné testovat, provozovat i rozvíjet bez přímé závislosti na konkrétní technologii. V praktickém smyslu to znamená, že doménová pravidla nesmějí být uzamčena v řadiči požadavků (controller), v rámci pro zpracování HTTP požadavků ani v databázovém ovladači.

Klíčová asymetrie této architektury neleží mezi „horní“, prezentační a „spodní“ datovou vrstvou, ale mezi vnitřkem a vnějškem aplikace [20]. Port v tomto pojetí nepředstavuje síťový port, nýbrž účelově definovaný kontrakt komunikace mezi jádrem a jeho okolím. K jednomu portu přitom může existovat více adaptérů, například produkční HTTP vstup, konzolový přístup nebo náhradní implementace trvalého uložení dat pro integrační testy [20]. Tento přístup je důležitý ze tří důvodů. Zaprvé udržuje víceorganizační kontext pod kontrolou od vstupu do systému až po datovou vrstvu. Zadruhé chrání doménu před přímou závislostí na integračních detailech, které se mohou v čase měnit. Zatřetí vytváří čitelnou strukturu, kterou lze dlouhodobě rozvíjet i bez velkého specializovaného architektonického týmu.

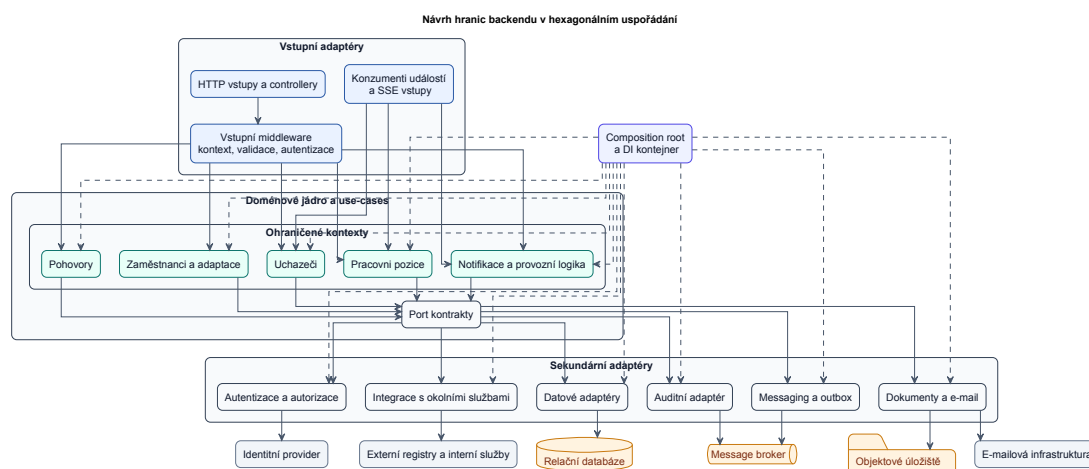
V terminologii této práce proto označuji adaptéry, které aplikaci řídí zvenku dovnitř, jako primární adaptéry, zatímco adaptéry vykonávající požadavky jádra směrem do infrastruktury označuji jako sekundární adaptéry. Primární adaptér převádí vnější signál na volání případu užití (use case), kdežto sekundární adaptér implementuje port definovaný jádrem a překládá jeho požadavek do konkrétního protokolu, rozhraní nebo technologie trvalého uložení. Současně je důležité odlišit architektonickou hranici od hranice nasazovací. Hexagonální architektura sama o sobě neznámá, že každý adaptér musí být samostatná služba. Naopak většina adaptérů běží uvnitř jednoho backendového procesu a samostatně oddělují jen ty části, jejichž běhový profil, provozní charakteristiky nebo integrační režim se od jádra skutečně liší.

Hexagonální uspořádání v mém návrhu současně doplňují principy doménově řízeného návrhu (DDD). Doménové moduly jsou vedeny jako ohraničené kontexty s vlastním modelem a slovníkem, protože význam pojmů je v doménovém návrhu vždy platný jen uvnitř explicitně vymezené hranice [22, 23]. Na styku s vnějšími službami proto integrační vrstva plní roli překladové mezivrstvy. Doména pracuje se svým modelem, zatímco adaptér přebírá odpovědnost za převod z cizího rozhraní a zpět [23].



Obrázek 4. 3: Obecný princip hexagonální architektury

Obecný princip zachycený na Obrázek 4. 3 se v backendu promítá do konkrétní struktury. Primární adaptéry na vstupu aplikace převádějí HTTP požadavky a integrační události na případy užití domény. Uvnitř aplikace zůstává doménová vrstva členěná do ohraničených kontextů a komunikuje s okolím pouze prostřednictvím explicitních kontraktů portů. Sekundární adaptéry pak zajišťují napojení na databázi, vrstvu předávání zpráv, audit, autorizaci a vnější registry.



Obrázek 4. 4: Návrh hranic backendu v hexagonálním uspořádání

Obrázek 4. 4 neslouží jako inventář implementačních složek, ale jako návrh hlavních hranic backendu. Ukazuje, že vstupní adaptéry řídí vstup do systému, doménové jádro nese případy užití a ohraničené kontexty, porty vymezují povolené závislosti a sekundární adaptéry připojují databázi, vrstvu předávání zpráv, audit, bezpečnost a okolní služby. Smyslem tohoto rozdělení je udržet stabilnější doménovou logiku uvnitř a proměnlivější integrační infrastrukturu na hraně systému.

4.4 DATOVÝ MODEL A INTEGRAČNÍ TOKY

Architektonická rozhodnutí by zůstala neúplná, pokud by se nepropsala i do datového modelu. Reálné schéma obsahuje přibližně šedesát tabulek a řadu vazeb, pro hlavní výklad je však důležitější jeho konceptuální kostra než úplný inventář všech struktur. V této části proto nepopisují všechny tabulky, ale vysvětlují, jakými objekty systém zachycuje organizační rámec, nábor, přijetí a adaptaci pracovníka.

Nejvyšším rámcem modelu je organizace. Každý klíčový záznam je k některé organizaci vztažen přímo nebo nepřímo, takže víceorganizační členění není jen pravidlo v aplikaci, ale vlastnost samotných dat. Vedle organizace stojí interní uživatel jako nositel identity. Jeho globální role říká, jaký typ uživatele v systému vystupuje, ale sama o sobě neurčuje, ke kterým datům smí přistoupit. To zajišťuje až členství v organizaci, které vyjadřuje vztah ke konkrétnímu závodu, a evidence oprávnění ke zdrojům, která jemněji vymezuje přístup k pracovním pozicím a k navázaným záznamům. Právě tato kombinace identity, členství a oprávnění umožňuje oddělit data jednotlivých závodů a současně zachovat centrální dohled.

V náborové oblasti je nutné odlišit pracovní roli a pracovní pozici. Pracovní role představuje obecný typ místa, například druh profese nebo zařazení. Pracovní pozice naproti tomu představuje konkrétní otevřené místo vypsání jednou organizací v určitém čase. Právě pracovní pozice je hlavním procesním objektem náboru. Nese vazbu na organizaci, odkazuje na pracovní roli, může využívat předem definovaný adaptační postup a stává se bodem, k němuž se vztahují další data. Na pracovní pozici se vážou uchazeči a k jednotlivým uchazečům se dále vážou pohovory, změny stavů a další rozhodnutí v průběhu výběrového řízení. Jinými slovy: organizace vypisuje pracovní pozici, pracovní role ji typizuje, na pozici reaguje uchazeč a nad uchazečem se evidují pohovory a další náborové kroky.

Zlom mezi nábořem a následnou adaptací nenastává vznikem zcela nové evidence, ale přechodem v rámci téhož životního cyklu. Pokud je uchazeč přijat, systém jej nepovažuje za izolovaný historický záznam, ale převádí jej do evidence interního uživatele, respektive zaměstnance. Tím vzniká výslovná vazba mezi přihláškou, výběrovým řízením a následným nástupem do zaměstnání. Pro data je to důležité ze dvou důvodů. Jednak lze zpětně dohledat celý průběh od první reakce na inzerát až po adaptaci, jednak se při přechodu nepřerušuje organizační kontext ani návaznost oprávnění.

Samostatný doménový blok tvoří adaptace. Zde je nutné odlišit definici adaptačního postupu od jeho konkrétního průběhu. První rovina popisuje šablonu. Jaké kroky má nový pracovník splnit, v jakém pořadí, za jakých podmínek a jaké dokumenty mohou být požadovány. Druhá rovina zachycuje skutečný průběh adaptace konkrétního zaměstnance, tedy co už bylo splněno, co čeká na dopl-

nění a kdo za daný krok odpovídá. Adaptační proces má tedy podobu obecného postupu na jedné straně a jeho konkrétní instance na straně druhé. Dokumenty zaměstnance jsou v této oblasti vedeny jako samostatná, ale úzce navázaná skupina dat. Nejsou to jen přílohy, ale důkazy splnění konkrétních povinností v průběhu nástupu.

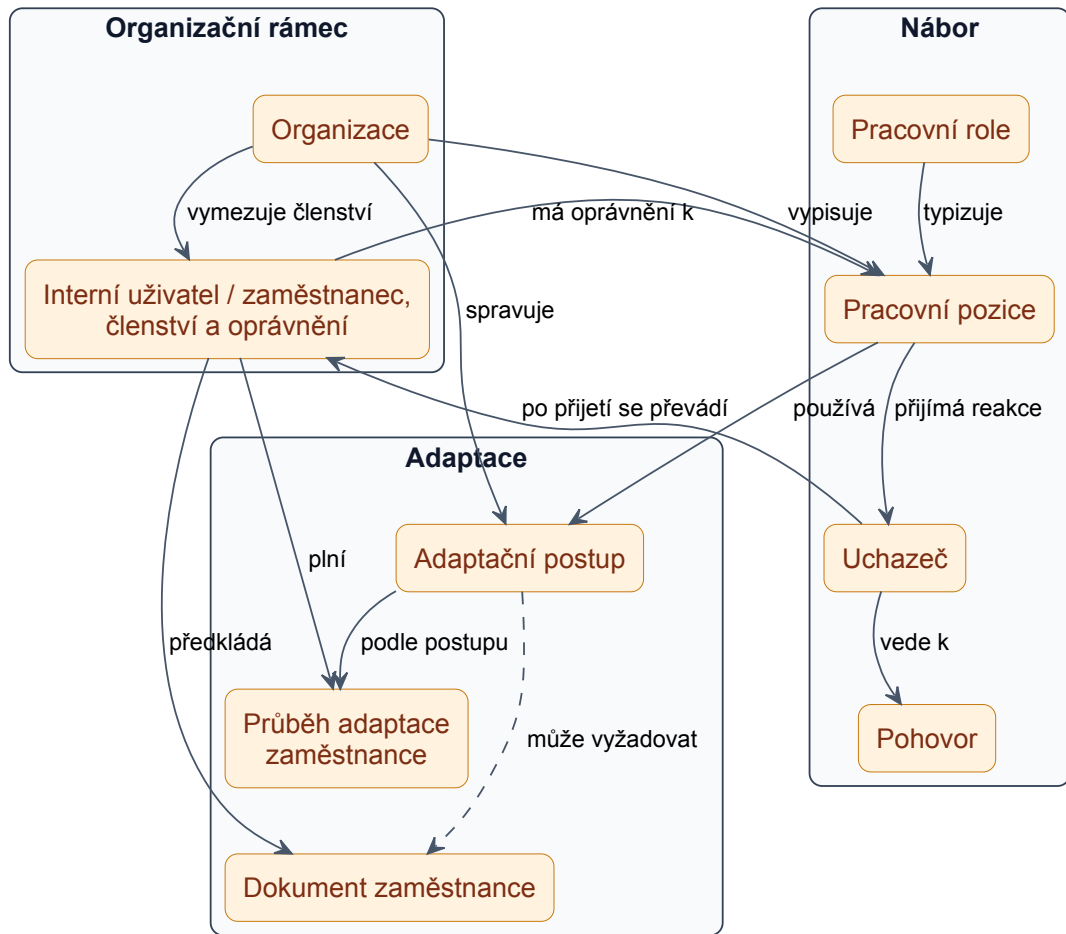
Vedle hlavního rámce systém obsahuje i doplňkovou větev zájemců o práci bez vazby na konkrétní pracovní pozici. Tato evidence má vlastní údaje, preference a přílohy, ale z hlediska modelu představuje vedlejší vstup do náboru, nikoli jeho hlavní větev. Podobně technické evidence, jako jsou soubory, auditní záznamy, asynchronní vedlejší účinky nebo analytické výsledky nad životopisy a pozicemi, nejsou jádrem konceptuálního modelu. Jsou pro provoz systému nezbytné, ale slouží především jako podpůrná vrstva nad hlavní doménou, a proto je zjednodušený diagram záměrně neukazuje. TODO: Tohle šoupnu spíše asi do implementace? nebo já nevím TODO

TODO Součástí datového návrhu však není jen to, co se ukládá, ale i jak dlouho má být která informace uchována. U neúspěšných uchazečů proto architektura nepočítá s neomezenou retencí osobních údajů. Po uplynutí zákonné nebo souhlasem vymezené lhůty se jejich data nestávají běžnou historickou evidencí, ale kandidátem na řízenou anonymizaci nebo výmaz. To je důležité i modelově. Identifikační a kontaktní údaje, přílohy a volné texty musí být oddělitelné od metadat o průběhu procesu tak, aby bylo možné odstranit osobní obsah a současně nezničit nutný provozní nebo statistický kontext.

Takový zásah nelze provádět ručně a izolovaně jen v jedné tabulce, protože údaje uchazeče zasahují do příloh, poznámek, auditní stopy i navazujících asynchronních úloh. Návrh proto předpokládá řízenou skartační operaci vedenou jako doménovou událost. Transakční vrstva označí záznam k anonymizaci nebo výmazu, vrstva odloženého odeslání zajistí navazující kroky nad souborovým úložištěm a dalšími službami a auditní vrstva uloží důkaz, že ke skartačnímu zásahu došlo, aniž by sama dále uchovávala původní osobní obsah. V auditní stopě tak zůstává informace o typu operace, čase a technickém identifikátoru zásahu, nikoli plná data, která měla být odstraněna.

Obrázek 4. 5 schematicky shrnuje výše popsané bloky a jejich nejdůležitější vazby. Jednotlivé obdélníky proto nepředstavují vždy jedinou fyzickou tabulku, ale spíše jednu část domény nebo skupinu úzce souvisejících entit.

Zjednodušený model hlavních datových entit



Obrázek 4. 5: Zjednodušený diagram hlavních entit a vztahů

Pružnou část modelu představují adaptační formuláře a odpovědi zaměstnanců, které ukládám jako jsonb. Důvodem je, že obsah těchto kroků se může měnit podle role, pracoviště i interních pravidel a čistě relační model by zde vedl k častým změnám schématu kvůli drobným úpravám formulářů. Výhodou je větší pružnost, omezením přesun části validačních pravidel do aplikační logiky.

Zvláštní roli mají také vektorové reprezentace životopisů a pracovních pozic, které ukládám pomocí pgvector přímo v PostgreSQL. Sémantické vyhledávání tak zůstává součástí stejného datového prostředí jako transakční agenda, což zjednodušuje správu a udržuje vazbu mezi běžnými daty a analytickými výstupy.

Na takto vymezený datový model přímo navazuje způsob, jakým systém komunikuje s okolními službami a jak provádí vedlejší události. Integrační vrstvu proto navrhuji jako kombinaci synchronní a asynchronní komunikace. Synchronní volání používám tam, kde uživatel očekává okamžitou odezvu, například při práci s administračním rozhraním nebo při generování konkrétního výstupu. Asynchronní komunikaci naopak využívám pro vedlejší události a časově náročnější operace, u nichž by blokování požadavku zhoršovalo použitelnost systému.

Podrobný průběh této kombinace, včetně vzoru odloženého odeslání (transactional outbox) a návaznosti na vrstvu inteligentního zpracování dat, rozvádím až v implementační kapitole. Cenou za tento model je složitější dohled nad tím, zda byly navazující události opravdu provedeny a v jakém pořadí. Právě proto je asynchronní komunikace v návrhu úzce svázána s auditní a dohledovou vrstvou.

Pro komunikaci směrem ke klientské aplikaci architektura počítá s mechanismem serverem zasílaných událostí (Server-Sent Events, SSE) pro průběžné informování o stavu operací. Ten umožňuje zobrazovat změny nebo postup zpracování bez nutnosti opakovaného dotazování ze strany klienta a zároveň nezatěžuje systém plně obousměrnou komunikací.

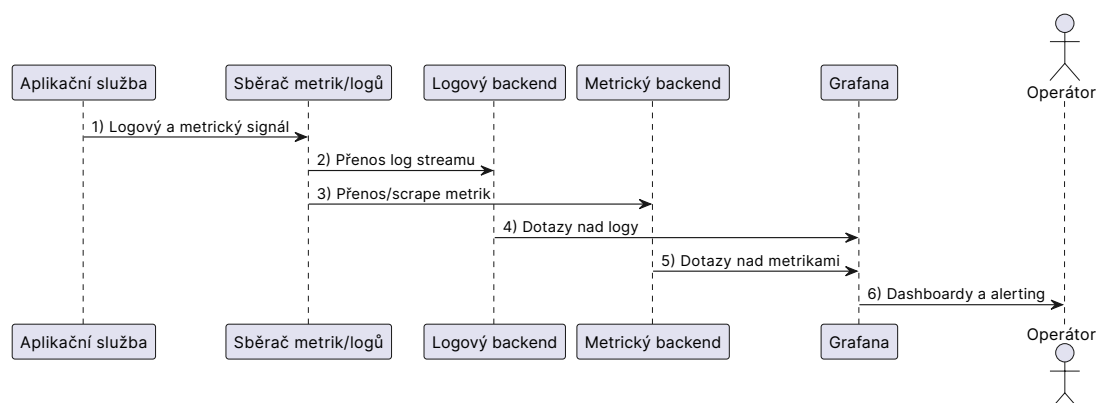
4.5 RÁMEC BEZPEČNOSTI A SPOLEHLIVOSTI

Jakmile systém propojuje více rolí, více závodů a citlivé personální údaje, stává se bezpečnostní architektura jedním z rozhodujících kritérií návrhu. Bezpečnost se tedy řeší ve třech vrstvách a to jako ověření totožnosti (autentizace), řízení přístupu (autorizace) a datový rozsah. Ověření totožnosti je řešeno prostřednictvím protokolu OpenID Connect (OIDC) s napojením na jednotné přihlašování (SSO) organizace, aby interní uživatelé nemuseli spravovat oddělené identity pouze pro tento systém. Tento krok zároveň snižuje provozní režii spojenou se správou účtů.

Řízení přístupu nestavím jen na klasickém modelu rolí (RBAC, Role-Based Access Control), ale na kombinaci globálních rolí a řízení přístupu na základě vztahů (ReBAC, Relationship-Based Access Control). Vedl mě k tomu proces, kde vedoucí pracovník může potřebovat přístup jen k jedné konkrétní pozici nebo skupině kandidátů, nikoli k celému závodu. Samotná role tak popisuje typ uživatele, zatímco vztah ke zdroji rozhoduje o skutečném rozsahu dat.

Součástí bezpečnostního návrhu je i princip nejnižších oprávnění a auditovatelnost operací nad citlivými daty (NF11). Změny nad klíčovými entitami musí být zpětně dohledatelné, a to nejen kvůli bezpečnosti, ale i kvůli schopnosti vysvětlit průběh náboru nebo adaptace při interním přezkumu.

Bezpečnostní vrstva však sama nestačí. V prostředí s lokální infrastrukturou je stejně důležité včas rozpoznat, že se systém začíná odchylovat od očekávaného chování. Proto architekturu uzavírá samostatná vrstva provozního dohledu. Kombinace nástrojů Promtail, Loki, Prometheus a Grafana umožňuje sběr protokolů, metrik i jejich společnou interpretaci z jednoho místa, aniž by bylo nutné zasahovat do doménové logiky systému.



Obrázek 4. 6: Tok dat v monitorovací vrstvě od aplikace po vizualizaci

Vedle samotného sběru dat navrhuji i dva typy výstražných mechanismů. Provozní výstrahy mají zachytit zhoršení dostupnosti nebo odezvy klíčových koncových bodů rozhraní. Výstrahy vázané na cílové úrovně služby (Service Level Objectives, SLO) pak sledují zdraví asynchronní vrstvy, například stáří nejstarší nevyřízené zprávy nebo počet zpráv, které selhaly po maximálním počtu pokusů. Smyslem této vrstvy není produkovat více přehledových panelů, ale dát provozu včasný signál, že se systém začíná odchylovat od očekávaného chování.

5 IMPLEMENTACE SYSTÉMU

Architektonický návrh má smysl jen tehdy, pokud se podaří převést jeho principy do konkrétní implementace bez toho, aby se po cestě rozpadly pod tlakem technologických kompromisů. V této kapitole proto neuvádím úplný výpis všech tříd, tabulek a endpointů, ale ta rozhodnutí, na nichž se láme rozdíl mezi formálně správným návrhem a skutečně provozně použitelným systémem.

Pozornost soustředím především na modulární backend, hexagonální rozdělení odpovědností, spolehlivou asynchronní komunikaci, datovou vrstvu, bezpečnostní model a oddělenou vrstvu inteligentního zpracování dat. Právě v těchto částech se ukazuje, zda navržená architektura dokáže unést reálné procesní a provozní požadavky KZ.

5.1 STRUKTURA ŘEŠENÍ

Celé řešení je implementováno jako sada spolupracujících aplikací a služeb s oddělenými odpovědnostmi. Backend je realizován v prostředí Node.js/Express, webové portály jsou postaveny na Next.js a podpůrné zpracovatelské či integrační služby běží převážně v jazyce Go.

Pro backend jsem zvolil Node.js/Express, protože řešené jádro je především API s velkým podílem HTTP komunikace, middleware logiky a integračních vazeb. Express současně ponechává přímou kontrolu nad skladbou aplikace, což bylo důležité pro promítnutí hexagonálního uspořádání do kódu. Alternativou byl NestJS nebo jiný backendový stack, například Spring Boot či ASP.NET Core, ty by však v daném rozsahu přinesly více konvencí a vyšší implementační režii bez zřetelného přínosu.

Webové portály jsem postavil na Next.js. Vedle technických vlastností zde hrála důležitou roli i vyzrálost ekosystému. Široká komunita, dobře dostupná dokumentace a průběžný vývoj frameworku snižují riziko, že se řešení dostane do technologicky okrajové větve s horší udržitelností. Současně jde o přirozené rozšíření React prostředí, takže bylo možné využít známý vývojový model i rozsáhlé sdílené know-how. Alternativou byla čistě klientská aplikace v React bez serverového vykreslení nebo řešení v jiném ekosystému, například Nuxt. Zvolený přístup však lépe podporoval dlouhodobou udržitelnost i sjednocení obou portálů v jednom technologickém základu.

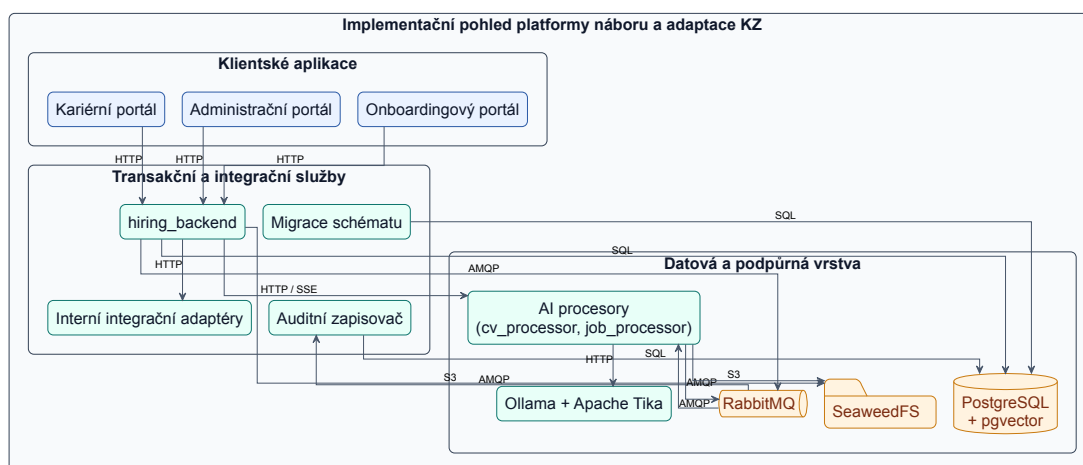
Volba jazyka Go u procesorů přitom nebyla motivována tím, že by právě tento jazyk nabízel zásadně lepší knihovny pro práci s modely. Samotná inference je

totiž delegována do Ollama a extrakce textu do Apache Tika, tedy do oddělených služeb dostupných přes HTTP. Z čistě funkčního hlediska by proto bylo možné procesory implementovat i v Node.js, což by sjednotilo technologický stack. Rozhodující byly spíše jejich provozní vlastnosti. `cv_processor` běží jako dlouhožijící worker s více souběžnými konzumenty fronty a `job_processor` jako samostatná HTTP/SSE služba. Go v tomto kontextu přináší jednoduchý model souběžnosti, malé samostatně nasaditelné binární soubory a předvídatelné chování u specializovaných služeb, které neřeší rozsáhlou doménovou logiku ani práci s uživatelským rozhraním.

Vedle hlavního backendu řešení zahrnuje i samostatné služby pro auditní zápis, integrační adaptéry a inteligentní zpracování dokumentů či textů pracovních pozic. Tyto komponenty spolu komunikují přes HTTP nebo přes RabbitMQ. RabbitMQ jsem zvolil proto, že umožňuje spolehlivě oddělit transakční cestu od vedlejších efektů, aniž by bylo nutné zavádět robustnější streamovací platformu. Alternativou bylo čistě synchronní volání mezi službami nebo platforma typu Kafka. První varianta by zhoršovala odezvu a odolnost systému, druhá by v tomto měřítku přinesla spíše vyšší provozní režii.

Zvolený technologický mix tedy nesměřuje k pestrosti pro ni samu, ale k přiřazení vhodného nástroje konkrétnímu typu problému. Přínosem je lepší přizpůsobení jednotlivých částí jejich provoznímu profilu, omezením naopak vyšší nárok na koordinaci buildů, nasazení a dohledu.

Obrázek 5. 1 ukazuje nasazované komponenty řešení a jejich hlavní komunikační vazby. Je na něm vidět trojice klientských aplikací nad společným backendem, návaznost backendu na integrační a auditní služby, propojení s databází, objektovým úložištěm a vrstvou předávání zpráv i oddělení inteligentní vrstvy od hlavní aplikační cesty.



Obrázek 5. 1: Nasazované komponenty řešení a jejich hlavní komunikační vazby

5.2 IMPLEMENTACE BACKENDU

Backend byl implementován jako modulární aplikace s hexagonálním architektonickým uspořádáním. Každý požadavek vstupuje do systému prostřednictvím routovací vrstvy a middleware pipeline, která zajišťuje autentizaci, autorizaci, vytvoření kontextu požadavku a jednotné mapování chyb. Po úspěšném průchodu touto infrastrukturní částí je požadavek předán vstupnímu adaptéru (např. HTTP controlleru), který jej převede na aplikační vstupní model a zavolá odpovídající případ užití (use-case) v aplikační vrstvě.

Samotné use-cases tvoří vstupní porty aplikace a určují, jaké operace systém poskytuje. Tyto use-cases orchestrují průběh operace, ověřují aplikační pravidla a delegují samotná doménová pravidla na doménové objekty a služby, aniž by je samy implementovaly. Pokud aplikace potřebuje komunikovat s okolním světem, činí tak prostřednictvím výstupních portů, jejichž konkrétní implementace poskytuje infrastrukturní vrstva pomocí technologických adaptérů (tj. komponent převádějících komunikaci mezi externími systémy a porty aplikace) pro zajištění perzistence dat, auditování, asynchronního předávání zpráv, autentizace a externích integrací.

Skládání těchto částí řídí dependency injection (DI) kontejner Awilix. Volba tohoto nástroje byla podřízena snaze o maximální čistotu doménové vrstvy. Na rozdíl od alternativ, jako je například InversifyJS, nevyžaduje Awilix použití dekorátorů přímo v doménovém kódu. Tím je dosaženo stavu, kdy doménová logika zůstává technologicky agnostická a neobsahuje žádné závislosti na použitém DI frameworku, což odpovídá principům hexagonální architektury. Awilix byl dále zvolen pro svou podporu tzv. request scoping, který umožňuje v rámci jednoho požadavku sdílet kontext (např. identitu uživatele či databázovou transakci) napříč vrstvami bez nutnosti jejich explicitního předávání.

Takto navržený backend plní roli autoritativní transakční hranice systému. Právě zde se rozhoduje o tom, zda je konkrétní změna v souladu s doménovými pravidly, s rozsahem oprávnění uživatele i s požadavky na auditní dohledatelnost. Z formálního hlediska jde o koncentraci invariantů, tedy pravidel, která musí v systému vždy platit, do doménové a aplikační vrstvy, což minimalizuje riziko jejich nekonzistentní implementace napříč frontendem, integračními službami a databázovými skripty.

Pro lepší přehlednost o rozdělení odpovědností v rámci systému je níže uvedena tabulka Tabulka 5. 1, která mapuje teoretické prvky hexagonální architektury na jejich konkrétní realizaci v rámci této implementace.

Tabulka 5. 1: Mapování prvků hexagonální architektury na implementační vrstvy backendu

Prvek hexagonu	Implementační realizace
Vstupní adaptéry	Routovací vrstva, middleware a řadiče (controllers) převádějící HTTP požadavky na případy užití
Aplikační vrstva / vstupní porty	Případy užití (use-cases), které definují operace poskytované aplikací a orchestrují jejich průběh
Doménové moduly	Entity, value objekty, doménové služby a doménové události ohraničených kontextů (bounded contexts)
Výstupní porty	Explicitní rozhraní (kontrakty), kterými aplikační nebo doménová vrstva vyjadřuje požadavky na okolní systémy
Výstupní adaptéry	Konkrétní implementace perzistence, auditu, asynchronního předávání zpráv, úložiště a externích či interních integrací
Vazba port-adaptér	Kompoziční kořen (composition root) a DI kontejner Awilix, které propojují porty s jejich konkrétními adaptéry

5.2.1 POUŽITÉ VZORY A TECHNIKY V BACKENDU

Tato kapitola se zaměřuje na nejdůležitější klíčové návrhové vzory a techniky, které byly při vývoji backendu aplikovány pro zajištění jeho dlouhodobé udržitelnosti, čitelnosti a provozní spolehlivosti. Prezentované vzory slouží jako praktické řešení opakujících se architektonických výzev, jež přirozeně vznikají v komplexních systémech propojujících doménovou logiku s databázovými operacemi, asynchronním zpracováním zpráv a externími vazbami. Každý popsany vzor je tak zasazen do kontextu konkrétního problému, který v aplikaci efektivně řeší.

Prvním problémem je riziko, že se doménová logika postupně smísí s HTTP vrstvou, databázovými detaily a integračním kódem. Tento problém řeší hexagonální uspořádání. Jeho smyslem je udržet rozhodování o nábore, zaměstnancích, oprávněních nebo adaptaci uvnitř aplikace a všechny proměnlivější technologické detaily vytlačit na její okraj. Prakticky to znamená, že doménová služba neřeší, zda se data načítají přes SQL, zda se požadavek na kvalifikaci převádí na interní HTTP volání nebo zda se vedlejší efekt doručuje přes AMQP a outbox. To je totiž práce primárních a sekundárních adapterů. Přínosem je, že změna databázového

adaptéru nebo integrační služby nevede sama o sobě k přepisu doménových pravidel. Omezením je vyšší počet explicitních kontraktů a větší nárok na kázeň při návrhu hranic.

Druhým problémem je vazba aplikačních služeb na fyzické schéma databáze. Pokud by use-cases pracovaly přímo s SQL dotazy a tabulkovými detaily, začala by se datová struktura propisovat do míst, která mají rozhodovat o business pravidlech. Tento problém řeší vzor Repository. Jeho úlohou je zapouzdřit datový přístup tak, aby aplikační vrstva pracovala s operacemi, které dávají význam z pohledu domény, nikoli z pohledu konkrétní tabulky. Přínosem je čitelnější odpovědnost jednotlivých částí a menší citlivost business logiky na změny perzistence. Omezením je další úroveň abstrakce, kterou je třeba udržovat konzistentně.

Dalším, v pořadí třetím, problémem je skládání závislostí v aplikaci, která už není malým monolitem, ale systémem s porty, adaptéry, službami a specializovanými moduly. Bez řízeného skládání by jednotlivé části backendu začaly samy vytvářet konkrétní implementace svých závislostí a architektonické hranice by se postupně rozpadly. Tento problém řeší Dependency Injection, v mém případě realizovaná kontejnerem Awilix. Přínosem je možnost měnit implementace adaptérů bez zásahu do volající logiky a současně snazší testovatelnost. Omezením je, že část vazeb vzniká až při skládání aplikace, takže chyba v registraci se projeví později než u čistě přímých závislostí.

Čtvrtým problémem je vazba vedlejších efektů na transakční zápis. Backend často potřebuje po úspěšném uložení dat rozeslat notifikaci, vytvořit auditní událost nebo publikovat úlohu pro další zpracování. Pokud by se tyto kroky volaly přímo v jedné request cestě, vznikalo by riziko, že se business data uloží, ale navazující krok selže a systém zůstane v neúplném stavu. Tento problém řeší Transactional Outbox. Doménová operace proto ukládá business změnu i požadavek na vedlejší efekt v jedné transakci a teprve následná zpracovatelská vrstva zajistí skutečné doručení. Přínosem je vyšší konzistence mezi databází a integrační vrstvou. Omezením je vyšší provozní složitost a potřeba sledovat stav front a stáří nevyřízených položek.

Na outbox přímo navazuje problém duplicitního provedení. V asynchronním nebo síťově nestabilním prostředí nelze předpokládat, že každý požadavek nebo každá zpráva dorazí právě jednou. Systém proto musí umět rozlišit nový zápis od opakovaného doručení stejné operace. Tento problém řeší Idempotency. Přínosem je odolnost vůči opakovanému kliknutí, retrieable chybám nebo znovudoručení zprávy. Omezením je potřeba vést pomocná data o již zpracovaných požadavcích a přesně vymezit, které operace se mají chovat idempotentně.

Posledním důležitým problémem je kontakt s vnějšími systémy, jejichž datový model, protokol i chybové chování backend nekontroluje. Přímé napojení domény na tyto služby by vedlo k tomu, že cizí integrační detaily začnou určovat tvar interní logiky. Tento problém řeší adaptér, přesněji integrační mezivrstva blízka

Anti-Corruption Layer. V mém řešení se tento přístup uplatňuje například u vyhledávání kvalifikací nebo interních uživatelů. Přínosem je ochrana domény před proměnlivostí cizích rozhraní a možnost měnit integrační detail bez zásahu do jádra systému. Omezením je další překladová vrstva, kterou je nutné navrhovat a testovat stejně pečlivě jako samotnou doménu.

Backend tak nestojí na jednom izolovaném vzoru, ale na jejich součinnosti. Hexagonální uspořádání chrání doménu před infrastrukturou, Repository omezuje přímé prorůstání databázových detailů do aplikační logiky, Dependency Injection drží pod kontrolou skládání závislostí, Transactional Outbox a Idempotency stabilizují zápisové a integrační toky a integrační adaptéry oddělují cizí systémy od vnitřního modelu. Teprve v této kombinaci dávají zvolená řešení smysl jako celek.

5.2.2 IMPLEMENTACE DOMÉNOVÉ VRSTVY

Doménové jádro je členěno do modulů odpovídajících hlavním kontextům systému, tedy zejména náboru, pracovním pozicím, pohovorům, zaměstnancům, organizacím, číselníkům a interním identitám. Každý modul má vlastní use-cases, repozitáře a události, takže změna v jedné oblasti nemusí automaticky rozbít zbytek systému.

Toto členění řeší dva problémy současně. Zaprvé omezuje přímé závislosti mezi nesouvisejícími částmi backendu. Zadruhé umožňuje číst a rozvíjet konkrétní oblast bez nutnosti držet v hlavě celou aplikaci. Právě tato vlastnost je důležitá pro dlouhodobou udržitelnost projektu.

5.2.3 VYNUCENÍ ARCHITEKTONICKÝCH PRAVIDEL

Architektonická pravidla nestačí pouze deklarovat. Proto jsem je v implementaci částečně vynutil testy architektury. Ty ověřují, že doménové moduly neimportují repozitáře jiných modulů napřímo, že služby nepoužívají infrastrukturu mimo definované porty a že business vedlejší efekty nejsou volány mimo outbox handlers.

Tento mechanismus je důležitý hlavně proto, že architektonická kázeň má tendenci upadat právě v okamžiku, kdy projekt roste a přibývá tlak na rychlé úpravy. Testy zde fungují jako technická pojistka proti tomu, aby se výjimka z pravidla stala novým standardem.

5.2.4 IMPLEMENTACE SPOLEHLIVÉ ASYNCHRONNÍ KOMUNIKACE

Klíčovým implementačním problémem bylo zajistit, aby vedlejší efekty navázané na doménovou transakci byly spolehlivě doručeny i při chybách integrační vrstvy a současně nebyly prováděny duplicitně. Tento problém řeším kombi-

nací vzorů Transactional Outbox a Idempotency, konkrétně tabulkou `side_effect_outbox`, samostatným workerem a řízením zápisových toků nad idempotency vrstvou. Doménová operace zapisuje business data i záznam do outboxu v jedné transakci až po commit fázi worker položku vyzvedne a publikuje.

RabbitMQ zde neplní pouze roli technického transportu, ale architektonického oddělovače mezi producentem a konzumentem události. Umožňuje časově oddělit transakční cestu požadavku od zpracování vedlejších efektů, vyrovnat krátkodobé špičky a izolovat lokální selhání konzumentů. Výměnou za to systém přijímá sémantiku opakovaného doručení a potřebu pracovat s eventual consistency. Právě proto musí být konzumenti navrženi idempotentně a provoz doplněn o dohled nad frontami, retry politikami a mrtvými zprávami.

Tabulka 5. 2: Typy outbox událostí implementovaných v backendu

event_type v outboxu	Implementační efekt
<code>email.welcome.v1</code>	Odeslání uvítacího e-mailu novému zaměstnanci
<code>email.raw.v1</code>	Odeslání generického e-mailu podle doménové události
<code>notification.role.v1</code>	Role-based fan-out notifikace v rámci organizace
<code>notification.user.v1</code>	Cílená notifikace konkrétnímu uživateli
<code>cv.publish.applicant.v1</code>	Publikace události pro AI analýzu CV uchazeče do RabbitMQ
<code>cv.publish.job_seeker.v1</code>	Publikace události pro AI analýzu CV zájemce
<code>job.embedding.requested.v1</code>	Publikace požadavku na AI zpracování job embeddingu

Worker používá dávkové zpracování, zamykání položek, řízené opakování a mrtvý stav pro neobnovitelné chyby. Přímé volání business vedlejších efektů je omezeno na outbox handlers; doménové služby pouze zapisují požadavek do outboxu. Tím se snižuje riziko nekonzistence mezi databází a integrační vrstvou.

Verzování událostí pomocí suffixu `v1`, případně dalších verzí, slouží k bezpečné evoluci integračního rozhraní. Pokud se změní struktura zprávy nebo její sémantika, lze zavést novou verzi bez nutnosti rozbít stávající konzumenty v jednom kroku.

Cena za tuto robustnost spočívá ve vyšší implementační i provozní komplexitě. Systém musí evidovat stav zpráv, řešit opakované zpracování, sledovat stáří

front a rozlišovat dočasnou chybu od neobnovitelného selhání. Přínosem je však výrazně vyšší konzistence dat a nižší riziko tichého výpadku vedlejších efektů, což je v auditovatelném informačním systému důležitější než minimální počet infrastrukturních komponent.

5.2.5 IMPLEMENTACE AUDITNÍ VRSTVY

Auditní stopu nezapisuji synchronně v hlavní request cestě, ale přes samostatný worker `audit_writer_processor`. Ten konzumuje auditní události z RabbitMQ, validuje jejich payload a ukládá výsledek do tabulky `audit_events` v PostgreSQL. Tím zkracuji odezvu aplikačního API a současně snižuji riziko, že dočasné selhání transportní nebo databázové vrstvy zablokuje běžný transakční tok.

Z implementačního hlediska je důležité odlišovat vykonávací komponentu od datové struktury. `audit_writer_processor` představuje samostatnou zpracovatelskou službu navázanou na messaging vrstvu, zatímco `audit_events` je perzistentní tabulka, do níž se auditní stopa ukládá. Tato dvojice společně zajišťuje, že audit zůstává mimo kritickou request cestu, ale neztrácí vazbu na transakční dění systému.

Takto navržená auditní vrstva řeší napětí mezi dvěma požadavky, které se v personálních systémech často dostávají do konfliktu: vysokou dohledatelností a přijatelnou odezvou aplikace. Přínosem je, že audit nezpomaluje každou zapisovací operaci a lze jej dále zpracovávat jako samostatný datový tok. Omezením je krátké časové okno mezi vznikem události a jejím perzistentním zápisem, a tedy i nutnost provozně sledovat zdraví auditního kanálu stejně pečlivě jako samotného transakčního API.

5.3 IMPLEMENTACE VRSTVY INTELIGENTNÍHO ZPRACOVÁNÍ DAT

Vrstva inteligentního zpracování dat vznikla jako řešení na provozní tlak popsany v analytické části práce. V prostředí KZ probíhá nábor kontinuálně, takže systém souběžně pracuje s větším množstvím životopisů i průběžně upravovaných pracovních nabídek. Manuální screening životopisů je přitom časově náročný a snadno vede k přehlédnutí důležitých údajů. Transakční jádro proto nemá nést dokumentové a AI úlohy přímo.

Na tuto situaci odpovídám oddělenou vrstvou implementovanou jako samostatné služby v jazyce Go. Komunikace probíhá asynchronně přes RabbitMQ a inference zajišťuje Ollama s konfigurovatelným generativním modelem podle konkrétní služby a nasazení. Smyslem této vrstvy není řídit samotný náborový proces, ale doplnit jej o podpůrné schopnosti tam, kde by přímé zpracování dokumentů a textů zbytečně zatěžovalo backend.

Služba `cv_processor` zpracovává příchozí životopisy a je navázána na asynchronní tok přes RabbitMQ. Po přijetí události stáhne dokument z objektového úložiště, extrahuje text přes Apache Tika, pomocí Ollama provede strukturované vyčtení a shrnutí a nad připraveným textem vytvoří embedding pomocí modelu `nomic-embed-text`. Výsledek pak vrací zpět do backendu přes RabbitMQ. Embeddingy zde neslouží jen jako doplňkové metadata, ale jako podklad pro sémantické porovnávání uchazečů a pozic. Backend je ukládá do `pgvector` jako `vector(768)`.

Služba `job_processor` řeší jiný typ úlohy. Vystupuje jako samostatná HTTP/SSE služba, kterou `hr-backend` volá synchronně při generování a úpravě textů pracovních nabídek. Také zde se používá Ollama, ale role služby je odlišná než u `cv_processor`; zatímco jedna služba zpracovává dokumenty a vrací strukturované výstupy do asynchronního toku, druhá obsluhuje interaktivnější tvorbu textu blízkou uživatelské operaci.

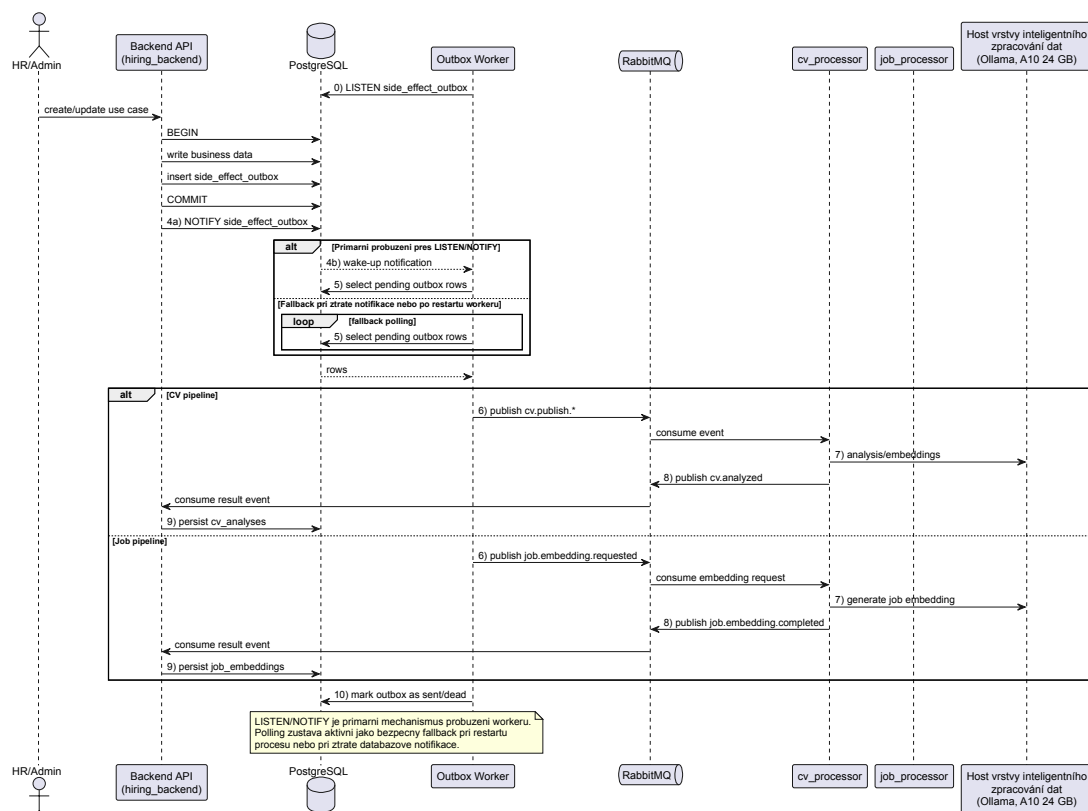
Vedle toho běží samostatný asynchronní tok pro embeddingy pracovních pozic. Backend publikuje požadavek `job.embedding.requested.v1`, zpracování připraví text pozice do podoby vhodné pro vektorové porovnávání a výsledný embedding se vrací do tabulky `job_embeddings`. I zde se používá `nomic-embed-text`, takže sémantické porovnávání pracuje nad stejným typem reprezentace jako u životopisů. Oba procesory tak sdílejí stejný inferenční backend, ale plní odlišné implementační role.

Z pohledu ochrany osobních údajů je důležité, že se nevyužívá externí cloudová AI služba. Ollama je provozována jako self-hosted inferenční vrstva v rámci on-premise infrastruktury a procesory ji volají lokálně přes interní rozhraní hostitelského serveru. Dokument životopisu, extrahovaný text i výsledné embeddingy se tak pohybují pouze mezi interním objektovým úložištěm, interní vrstvou zpráv, procesory a lokálně provozovaným modelem. Data proto neopouštějí provozní hranici organizace, což je pro prostředí KZ podstatný argument z hlediska ochrany osobních údajů a minimalizace regulatorního rizika. Zároveň je zpracování životopisů navázáno na evidenci souhlasu se zpracováním osobních údajů v náborovém toku.

Technické oddělení této vrstvy má přímý provozní důvod. Inference, práce s dokumenty a vektorové výpočty mají jiný latencní i chybový profil než transakční API. Jejich dočasná nedostupnost proto nesmí zablokovat základní náborové a onboardingové funkce. Systém tak degraduje řízeně. Uživatelé mohou přijít o část rozšířené funkcionality, nikoli o samotné transakční jádro.

Současně jde o ukázkou řízené distribuce pouze tam, kde přináší prokazatelný přínos. Oddělené procesory snižují tlak na backend a umožňují volit jiný runtime i škálování, ale zároveň zvyšují integrační složitost, potřebu observability a počet míst, kde může vzniknout prodleva nebo chyba přenosu. V implementaci proto tuto vrstvu oddělují jen pro úlohy s odlišným výpočetním profilem, nikoli jako obecné pravidlo pro celý systém.

Návaznost mezi transakční operací, vrstvou odloženého odeslání a zpracovatelskými službami shrnuje Obrázek 5. 2. Smyslem schématu není znovu popsat architekturu jako celek, ale ukázat, jak se v implementaci propojuje commit business dat, vznik outbox událostí, její předání do RabbitMQ a následné zpracování v oddělené AI vrstvě.



Obrázek 5. 2: Realizační tok Outbox-RabbitMQ a vrstvy inteligentního zpracování dat v implementaci

5.4 IMPLEMENTACE DATOVÉ VRSTVY A MIGRACÍ

Perzistenci dat stavím na PostgreSQL 17 s rozšířením pgvector. Relační část pokrývá transakční agendu náboru a onboarding, vektorová část podporuje scénáře inteligentního zpracování dat. Toto spojení řeší problém, jak držet konzistentně byznysová data i sémantické reprezentace bez zavádění další samostatné databázové technologie.

Volba PostgreSQL přitom nebyla vedena jen jeho rozšířením pgvector. Pro řešení typ systému je podstatné, že kombinuje silné transakční vlastnosti, referenční integritu a vyzrálý relační model s možností pracovat i s pružnějšími strukturami dat. Nábor, přijetí zaměstnance, auditní stopa i outbox obsahují vazby, které musí být konzistentní napříč více tabulkami a více kroky jedné

operace. Právě zde je důležitá schopnost spolehlivě provádět atomické zápisy, vynucovat cizí klíče a držet celý životní cyklus procesu nad jedním autoritativním zdrojem pravdy.

Současně jde o databázovou platformu, která v jednom prostředí pokrývá i požadavky, jež by jinak často vedly k nasazení více odlišných technologií. `jsonb` umožňuje ukládat proměnlivé struktury adaptačních formulářů bez zbytečné fragmentace schématu, `pgvector` rozšiřuje databázi o sémantické vyhledávání a standardní provozní vlastnosti PostgreSQL podporují zálohování, migrace i dohled v on-premise režimu.

V implementaci důsledně nesu organizační izolaci přes `organization_id` v klíčových entitách a kontrolují přístupový rozsah v aplikační vrstvě. Auditní stopu ukládám do samostatné struktury s omezením mutací, aby bylo možné doložit práci s citlivými daty i zpětně.

Volba jedné primární databázové platformy podporuje nejen provozní jednoduchost, ale i datovou konzistenci. Multi-tenantní filtr, auditní události, outbox i běžná transakční agenda jsou tak uloženy v jednom prostředí a lze je spravovat společně v rámci stejných zálohovacích a migračních postupů. Alternativou by bylo vyčlenit embeddingy do specializované vektorové databáze, například typu Qdrant nebo Pinecone, která by přinesla vyšší specializaci a potenciálně lepší škálování podobnostního vyhledávání. V daném měřítku řešení by to však znamenalo další datový subsystém, synchronizaci mezi transakční a vektorovou vrstvou, samostatné zálohování a větší integrační plochu v on-premise provozu. Volba `pgvector` byla proto jednoduchá.

5.4.1 FYZICKÝ DATOVÝ MODEL

Na úrovni fyzického modelu zachovávám stejnou logiku, která byla popsána v architektonické kapitole. Náborová část je vystavěna kolem vazby pracovní role → pracovní pozice → uchazeč → pohovor a navázané přílohy či změny stavů. Onboardingová část na ni navazuje oddělením šablony adaptačního postupu od jeho konkrétní instance nad zaměstnancem, takže databáze rozlišuje mezi definicí kroků a jejich skutečným plněním. Smyslem tohoto rozdělení není „mít více tabulek“, ale zabránit tomu, aby se jeden zápis současně tvářil jako definice procesu i jeho vykonaná instance.

Přístupový model je fyzicky opřen o `users`, `user_roles`, `organization_memberships` a `resource_permissions`. Datová vrstva tak přímo nese organizační kontext i pravidla ReBAC, místo aby byla autorizace řešena jen dodatečně nad již načtenými daty.

Konkrétní datové typy volím podle charakteru uložených informací. Formuláře a odpovědi zaměstnanců ukládám jako `jsonb`, protože jejich struktura se může v čase měnit. Embeddingy životopisů a pracovních pozic ukládám jako

vector(768) přes pgvector, aby bylo možné provádět sémantické porovnávání přímo v databázi.

5.4.2 MIGRACE SCHÉMATU

Schéma rozvíjím řízeně přes číslované SQL migrace. Tento přístup řeší problém, jak udržet databázový model auditovatelný a současně bezpečně nasaditelný do produkce. Běžné strukturální změny provádím transakčně standardními DDL příkazy, zatímco změny indexů pro živé prostředí odděluji do neblokujících kroků přes `CREATE INDEX CONCURRENTLY`.

Migrace zde zároveň fungují jako *quality gate* mezi novou binární verzí aplikace a stavem databáze. Backend tak nezačne běžet proti nekompatibilnímu schématu, což je v systému s více službami a asynchronními vazbami zásadní bezpečnostní pojistka.

Z teoretického pohledu představují migrace mechanismus řízené evoluce schématu, nikoli jen technický skript pro vytvoření tabulek. Umožňují zpětně doložit, kdy a proč se datový model změnil, a vytvářejí kontrolovaný přechod mezi verzemi systému. Omezením je nutnost disciplinovaně navrhovat i přechodové stavy, aby nová verze aplikace, dlouho běžící workery a stávající data nevytvářely nekompatibilní kombinace.

5.4.3 PERZISTENCE DOKUMENTŮ A INFRASTRUKTURNÍ TABULKY

Dokumenty uchazečů a zaměstnanců neukládám přímo do relační databáze. Tabulky jako `application_attachments`, `user_documents` nebo `onboarding_documents` obsahují metadata a reference na objektové klíče, zatímco fyzický obsah souborů je uložen v SeaweedFS přes S3 kompatibilní rozhraní. Tím řeším problém, jak zachovat výkonnou databázi pro transakční agendu a současně pracovat s většími soubory a jejich verzemi.

Vedle doménových tabulek používám i několik infrastrukturních tabulek. `side_effect_outbox` zajišťuje spolehlivé doručení vedlejších efektů po commit fázi, `command_idempotency` omezuje riziko duplicitního provedení zápisových operací a `audit_events` uchovává auditní stopu citlivých akcí. Tyto struktury nejsou byznysovou doménou samy o sobě, ale bez nich by nebylo možné naplnit požadavky na spolehlivost a auditovatelnost.

Stejná infrastruktura je důležitá i pro řízené nakládání s osobními údaji po uplynutí retenční lhůty. Výmaz nebo anonymizace neúspěšného uchazeče se v takovém systému nesmí omezit na prosté smazání jednoho řádku, protože osobní obsah bývá rozprostřen mezi databází, přílohami v objektovém úložišti a případnými odvozenými výstupy. Outbox proto poskytuje mechanismus, jak po rozhodnutí transakční vrstvy spustit navazující kroky konzistentně i mimo hlavní databázi,

zatímco auditní tabulka uchovává důkaz o provedení zásahu bez dalšího držení odstraněných osobních údajů.

5.4.4 IMPLEMENTACE BEZPEČNOSTI A IDENTITY

Bezpečnost je řešena ve více vrstvách. První vrstvou je autentizace uživatele přes session token, který middleware převádí na jednotný request kontext obsahující identitu, role a seznam organizací. Druhou vrstvou jsou endpoint guardy, které rozhodují, zda uživatel smí vstoupit do dané části API. Třetí vrstvou je datová autorizace v `hiring_backend`, realizovaná nad `resource_permissions` a návaznými vazbami na `organization_memberships`.

V praxi tím odděluji dvě různé otázky. Globální role uložená v `users.role_id` říká, jaký typ uživatele je přihlášen. Vztah ke konkrétní organizaci nebo pracovní pozici pak rozhoduje, k jakým datům skutečně smí přistoupit. `organization_memberships` proto neslouží jako zdroj role, ale jako evidence organizačních přístupů, jejich platnosti a způsobu přidělení.

Toto rozdělení řeší časté napětí mezi centralizovanou identitou a lokální datovou autorizací. SSO odpovídá na otázku, kdo je uživatel, zatímco backend musí samostatně rozhodnout, k jaké části multi-tenantního modelu má tento uživatel přístup. Přínosem je vyšší auditovatelnost a jemnější řízení oprávnění nad konkrétními zdroji. Omezením je vyšší složitost autorizačního modelu, který vyžaduje přesnou synchronizaci rolí, membershipů a odvozených oprávnění.

Tabulka 5. 3: Globální role a jejich praktický význam v ReBAC modelu backendu

Role	Praktický vztah k datům
Běžný uživatel systému	Pracuje primárně se svými scénáři a nemá zvýšená administrativní oprávnění
Read-only role pro náborový dohled	Vidí jen organizace, kde má membership, a konkrétní pozice, ke kterým má přímé přiřazení
Operativní práce s nábořem	Má <code>write</code> přístup k organizaci a pracovním pozicím ve svěřeném závodě
Správa organizace a přístupů	Má <code>admin</code> přístup v rámci své organizace a může spravovat role i membershipy
Systémová provozní role	Má globální administrativní rozsah napříč organizacemi i provozními moduly

Role check v middleware tedy není poslední autorizační krok, ale jen vstupní filtr. Samotné čtení, změny a mazání zdrojů vyhodnocuji až na úrovni práce s daty nad `resource_permissions`. Dědičnost oprávnění neřeším kopírováním

záznamů do každé dceřiné tabulky, ale relačním odvozením z nadřazeného zdroje. `resource_permissions` drží především oprávnění k organizaci a pracovní pozici, zatímco přístup k poznámkám uchazeče, jeho přílohám, pohovorům nebo přílohám pohovoru se vyhodnocuje přes vazbu na uchazeče a jeho `job_posting_id`. Pokud se tedy změní oprávnění k pracovní pozici, změní se automaticky i přístup ke všem navázaným dceřiným entitám bez nutnosti hromadně přepisovat jejich vlastní ACL záznamy. Přínosem je menší redundance a menší riziko zastaralých oprávnění. Omezením naopak složitější SQL dotazy a vyšší závislost na konzistenci relačních vazeb. Změny rolí a membershipů se navíc synchronizují asynchronně přes outbox události, aby model zůstal konzistentní i mimo kritickou request cestu.

5.5 IMPLEMENTACE ONBOARDINGOVÉHO PORTÁLU

Onboardingový portál jsem implementoval jako samostatnou aplikaci oddělenou od veřejného kariérního portálu. Toto rozdělení řeší problém, že HR pracovníci a nastupující zaměstnanci potřebují pracovat s odlišnými typy úloh i s jiným bezpečnostním režimem. Na úrovni implementace jsem proto oddělil layouty, cesty i stavovou logiku podle role uživatele.

Na implementační úrovni portál skládá pohled zaměstnance a interních pracovníků nad stejným backendem, ale s odlišnými layouty, trasami a stavovou logikou. Tím se zachovává jednotný zdroj pravdy v API a zároveň se respektuje, že HR role pracuje s přehledem procesu, zatímco nastupující zaměstnanec s konkrétní sadou kroků, termínů a dokumentů. Přínosem je vyšší srozumitelnost práce s onboardingem, omezením naopak nutnost udržovat další frontendovou aplikaci a synchronizovat její vývoj s backendem.

5.6 IMPLEMENTACE KARIÉRNÍHO PORTÁLU

Kariérní portál `kariera.kzcr.eu` jsem implementoval jako veřejný vstup do náborového procesu. Frontend je oddělen od backendu záměrně. Veřejný portál řeší prezentaci obsahu, navigaci a interakci s uchazečem, zatímco business pravidla pro práci s pozicemi, uchazeči a formulářovými daty zůstávají v jednom autoritativním API.

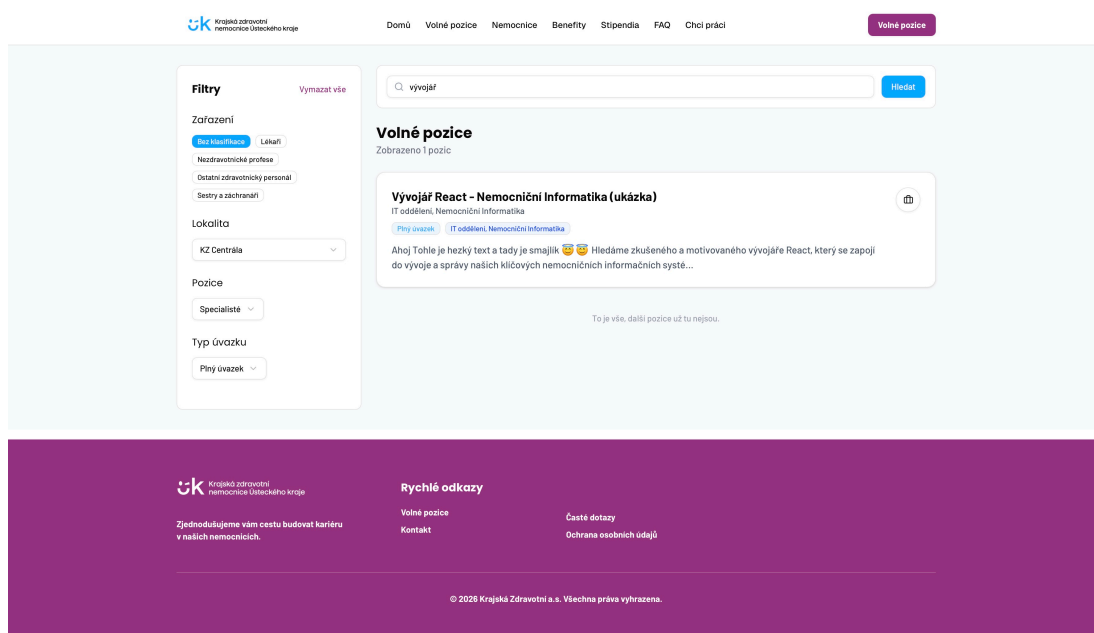
Domovská stránka propojuje obsahové a transakční scénáře. Uchazeč zde najde benefity, tematické kategorie pracovních rolí, mapový přehled nemocnic a vstup do katalogu volných míst. Samotný náborový tok tvoří seznam pozic, detail konkrétní nabídky a formulář reakce na vybranou pozici. Vedle toho portál obsahuje i samostatný kontakt pro zájemce bez vazby na konkrétní inzerát.

Komunikaci s backendem jsem soustředil do centralizované API vrstvy místo přímých HTTP volání z jednotlivých komponent. Tím řeším problém duplicitní síťové logiky v rozhraní a zjednodušuji změny v adresaci endpointů, timeoutu požadavků i mapování chybových stavů. Stav filtrů v katalogu pozic je navíc synchronizován s URL parametry, aby bylo možné konkrétní výběr sdílet a znovu otevřít ve stejném stavu.

Z pohledu uživatelské zkušenosti bylo důležité zachovat kontext při pohybu mezi seznamem a detailem pozice. Pro krátkodobý kontext používá portál `sessionStorage`, například pro návrat na stejné místo v seznamu, zatímco `localStorage` slouží k uchování vybraných pozic mezi jednotlivými navigacemi. Formulář reakce současně provádí klientskou validaci a používá idempotency klíč, aby se snížilo riziko duplicitních žádostí při opakovaném kliknutí nebo nestabilním spojení.

Vzhledem k tomu, že backend vrací popis pozice ve formátu HTML, řešil jsem i bezpečné vykreslení obsahu. HTML proto před zobrazením sanitizuji, aby se do stránky nedostal neověřený nebo škodlivý obsah. Jde o zdánlivý detail, ale právě na podobných místech se láme důvěryhodnost veřejného portálu.

Na Obrázek 5. 3 je zachycen katalog volných pozic v desktopovém zobrazení po aplikaci fulltextového dotazu a kombinace filtrů. V levé části rozhraní jsou ovládací prvky pro zpřesnění výběru, konkrétně zařazení, lokalitu, pracovní roli a typ úvazku, zatímco pravá část je vyhrazena pro vyhledávací pole, souhrn výsledku a samotný seznam nabídek. Toto rozvržení jsem zvolil proto, aby uchazeč mohl opakovaně upravovat dotaz bez ztráty kontextu a současně průběžně sledovat, jak se změna filtru promítá do výsledné množiny pozic.



Obrázek 5. 3: Katalog volných pozic s filtračním panelem a výsledkovou kartou

Rozhraní záměrně vychází z ustálených vzorců kariérních portálů. Náhledová karta shrnuje klíčové atributy nabídky a filtrační panel zůstává při práci s katalogem stabilní, aby uchazeč mohl porovnávat více výsledků bez opakované orientace v rozhraní. Nejde tedy o grafické rozhodnutí samo o sobě, ale o snahu snížit kognitivní zátěž v situaci, kdy uživatel rychle prochází větší množství podobných nabídek a rozhoduje se, zda pokračovat do detailu a formuláře reakce.

5.6.1 E2E TESTOVÁNÍ PORTÁLU

Spolehlivost veřejného portálu je zajištěna automatizovaným ověřením celé veřejné náborové cesty. Vzhledem k tomu, že portál představuje vstupní bod pro uchazeče, nepovažoval jsem za dostačující ověřovat jen jednotlivé prvky uživatelského rozhraní. Klíčové bylo chránit tok od vyhledání pracovní pozice přes otevření jejího detailu až po odeslání reakce s životopisem a současně tak ověřit integrační kontrakt mezi frontendem a odděleným transakčním API. Tento krok přímo podporuje naplnění nefunkcionálního požadavku NF04 na dostupnost systému, který požaduje dostupnost alespoň 99,5 % v pracovních dnech mezi 6:00 a 22:00, protože snižuje riziko, že se po změně frontendu nebo API naruší právě veřejně exponovaná náborová cesta. Jako alternativu jsem mohl zvolit pouze komponentové nebo integrační testy s mockovaným API, ty by však neodhalily poruchy vznikající až při běhu sestaveného portálu proti skutečně spuštěnému backendu. Reálnou alternativou byl i Cypress, avšak pro tento projekt jsem zvolil Playwright, protože umožňuje spouštět scénáře nad sestavenou verzí portálu v reálném prohlížeči, dobře pracuje s navigací, nahráváním příloh a více krokovým formulářovým tokem a současně se přirozeně integruje do CI/CD pipeline, včetně opakování selhaných běhů a záznamu trasování při chybě.

Testovací scénáře obsahují zobrazení veřejného seznamu pozic, vyloučení neveřejných nabídek, filtrování a vyhledávání nad skutečným API, přechod ze seznamu do detailu a dále na formulář reakce, odeslání přihlášky v režimu s povinným i nepovinným životopisem, klientskou validaci formulářů, samostatný kontaktní formulář i formulář pro zájemce bez vazby na konkrétní pozici. U klíčových scénářů nekončím na úrovni obrazovky, ale kontroluji i perzistenci výsledku v databázi, například vznik záznamu uchazeče, uložení přílohy nebo zápis kontaktního dotazu. Tímto postupem snižuji riziko, že po změně frontendu, API nebo validační logiky zůstane portál vizuálně dostupný, ale fakticky přestane spolehlivě přijímat použitelné reakce.

5.6.2 INTEGRACE ANALYTICKÉHO NÁSTROJE UMAMI

Pro analytické účely jsem do portálu integroval nástroj Umami. Jeho smyslem není zasahovat do rozhodování systému, ale měřit průchod uživatelů portálem a identifikovat místa, kde uchazeči proces opouštějí. Inicializace analytiky je pro-

vedena centrálně v kořenové komponentě aplikace, aby bylo zajištěno jednotné měření napříč stránkami.

Sleduji zejména interakce s katalogem pozic, otevření detailu nabídky, zahájení reakce na pozici, práci s formulářem a výsledek jeho odeslání. Tato data slouží jako podpůrný vstup pro další iterace portálu a doplňují kvalitativní poznatky z pilotního uživatelského ověření.

Umami jsem zvolil především proto, že odpovídá provozní logice celého řešení. Jde o relativně lehký nástroj, který lze provozovat ve vlastním prostředí, nevyžaduje rozsáhlou marketingovou konfiguraci a dobře se hodí pro měření účelově definovaných událostí nad veřejným portálem. Tato vlastnost je důležitá i ve vztahu k nefunkcionálnímu požadavku NF07, podle něhož musí být systém nasaditelný na infrastrukturu organizace v režimu on-premise. V mém případě bylo proto podstatné i to, že analytický skript mohu do aplikace připojit centrálně a při on-premise nasazení jej zpřístupnit přes vlastní proxy cestu, takže analytická vrstva nenarušuje architektonickou kontrolu nad veřejným rozhraním. Alternativou byly robustnější platformy typu Google Analytics 4, které však přinášejí vyšší závislost na externí cloudové službě a širší záběr, než jaký byl pro tento portál potřebný. Druhou realistickou alternativou bylo Matomo, které je rovněž možné provozovat lokálně, ale pro daný rozsah by znamenalo vyšší provozní režii a složitější správu. Umami se proto ukázalo jako přiměřený kompromis mezi kontrolou nad daty, jednoduchostí provozu a dostatečnou analytickou vypovídací hodnotou.

5.7 IMPLEMENTACE DOHLEDOVÉ VRSTVY

TODO: Ty půjdeš pryč a budeš jen v nasazení. Dohledovou vrstvu jsem implementoval jako kombinaci strukturovaného logování, metrik a centralizovaného dashboardingu. Tento krok řeší problém, že v distribuovanějším systému už nestačí číst lokální logy jednotlivých služeb. Potřebuji sledovat request kontext, zdravý asynchronní vrstvy i chování oddělených procesorů z jednoho místa.

Na úrovni aplikace generuji strukturované logy a publikuji metriky pro kritické toky, například autentizaci, outbox, messaging a vrstvu inteligentního zpracování dat. Tyto výstupy následně vstupují do samostatné monitorovací vrstvy popsané v architektonické kapitole.

5.8 KOMUNIKACE S NÁRODNÍMI REGISTRY

Napojení na národní registry jsem řešil primárně pro Národní registr zdravotnických pracovníků (NRZP) spravovaný ÚZIS. Praktická zkušenost ukázala, že samotné technické rozhraní ještě neznamená funkční integraci. Vedle klient-

ského certifikátu bylo nutné řešit i přidělení externích identifikačních údajů a odpovídajících oprávnění na straně poskytovatele služby.

Konkrétním integračním problémem bylo, že veřejně popsané SOAP/WSDL rozhraní neodpovídalo plně rozhraní, které bylo ve skutečnosti zpřístupněno pro provozní použití. V praxi se tak rozcházel očekávaný způsob napojení s reálně dostupnou vrstvou nad InterSystems IRIS, včetně konkrétních metod pro dotaz podle čísla pracovníka a rodného čísla. Součástí řešení proto bylo i doplnění potřebné WSDL popisné vrstvy a mapování metod na straně IRIS, aby bylo možné službu volat konzistentně a bez přenášení těchto odchylek do aplikační vrstvy. Nebylo proto účelné vázat backend přímo na generovaného SOAP klienta, protože by tím přebíral nestabilitu cizího integračního detailu do vlastní doménové logiky.

Z architektonického hlediska jsem proto integrační logiku neumístil přímo do `hiring_backend`, ale zarámoval ji jako adaptační mezivrstvu blízko vzoru `Adapter / Anti-Corruption Layer` a oddělil ji do interní služby `qualification-adapter`. Backend vystavuje pouze administrační endpoint `POST /api/v1/admin/qualifications/lookup`, který je dostupný vybraným rolím. Doménová služba podporuje dotaz podle čísla pracovníka v NRZP i podle rodného čísla, vstup normalizuje a validuje a teprve potom volá platformní adapter.

`qualification-adapter` běží pouze v interní Docker síti a funguje jako mezivrstva mezi backendem a integrační vrstvou nad InterSystems IRIS. Volba IRIS zde není náhodná. V prostředí KZ se tato platforma dlouhodobě používá jako integrační vrstva pro napojování okolních systémů, a proto bylo vhodnější navázat na existující provozní standard než zavádět pro jedinou vazbu samostatný integrační middleware. Tento mezistupeň pak řeší tři problémy najednou. Izoluje specifika skutečně dostupného integračního rozhraní mimo business logiku, sjednocuje chybové stavy do kontrolovaného HTTP kontraktu a zjednodušuje případnou výměnu integračního detailu bez zásahu do domény.

Výstup z registru backend převádí do jednotné doménové struktury obsahující identifikaci pracovníka a seznam odborných, specializovaných a zvláštních odborných způsobilostí. Současně vytváří auditní záznam o úspěšném i neúspěšném dotazu. Z důvodu ochrany osobních údajů se v auditu neukládá plné rodné číslo ani plné identifikátory dotazu, ale pouze jejich maskovaná podoba a hash. Implementace tak spojuje automatizaci kontroly kvalifikace s provozní kontrolou nad citlivou integrační vazbou.

6 NASAZENÍ SYSTÉMU DO CÍLOVÉHO PROSTŘEDÍ

Navržený systém má skutečnou hodnotu pouze tehdy, pokud je dlouhodobě provozně udržitelný v reálném prostředí, ve kterém bude nasazen. Klíčovou roli přitom hraje schopnost systém nejen jednorázově nasadit, ale opakovaně a spolehlivě reprodukovat jeho nasazení v různých prostředích. Stejně důležité je zavedení řízení změn, které umožňuje bezpečně provádět úpravy bez narušení stability provozu.

Neméně podstatná je schopnost systému včas signalizovat provozní odchylky, tedy situace, kdy se jeho chování odchyluje od očekávaného stavu. V praxi se může jednat například o prodlouženou dobu odezvy, zvýšenou chybovost, nedostupnost služby, neobvyklé zatížení infrastruktury nebo narušení komunikace mezi jednotlivými komponentami. Včasná identifikace těchto stavů je předpokladem pro jejich rychlou diagnostiku a minimalizaci dopadů na uživatele i navazující procesy.

Cílem této kapitoly je proto shrnout způsob, jakým bylo navržené řešení připraveno pro nasazení do cílového prostředí. Pozornost je věnována zejména kontejnerizaci jednotlivých částí systému, distribuci verzovaných obrazů, správě konfigurace, oddělení provozních zón a mechanismům umožňujícím dohled nad během systému.

6.1 KONTEJNERIZAČNÍ MODEL A SÍŤOVÁ SEGMENTACE

Jednotlivé části systému distribuují jako samostatné obrazy a jsou provozovány pomocí Docker Compose. Tento přístup odpovídá pilotnímu a on-premise charakteru řešení. Umožňuje zachovat oddělené běhové jednotky bez toho, aby bylo nutné zavést rozsáhlejší orchestraci už v první fázi projektu.

Provozní přínos kontejnerizace spočívá v tom, že stejný obraz může projít testem i cílovým nasazením bez dodatečného přestavování hostitelského prostředí. Tím se snižuje riziko rozdílů mezi prostředím a zjednodušuje se návrat ke starší verzi. Omezením naopak zůstává správa většího počtu obrazů, jejich průběžné bezpečnostní aktualizace a absence některých schopností, které poskytují robustnější orchestrátory.

Přehled služeb v Tabulka 6. 1 zachycuje provozní členění nasazení podle jejich role, stavového charakteru a míry expozice vůči okolnímu prostředí. Tabulka slouží jako orientační pohled na to, které komponenty tvoří cílové nasazení a jakou provozní odpovědnost v něm zastávají.

Tabulka 6. 1: Hlavní služby v nasazení

Název služby	Role	Stav	Expozice
kariera.kzcr.eu	Kariérní portál pro publikaci inzerátů a podání přihlášek	Bezstavová	Veřejná
onboarding.kzcr.eu	Onboardingový portál pro HR a zaměstnance	Bezstavová	Veřejná
hr-backend	Transakční API a integrační logika	Bezstavová	Interní
migration	Migrace databázového schématu	Bezstavová	Bez expozice
qualification-adapter	Vyhledání kvalifikací z NRZP	Bezstavová	Interní
user-search-adapter	Vyhledání interních uživatelů	Bezstavová	Interní
audit_writer_processor	Zápis auditních událostí	Bezstavová	Interní
cv_processor	Inteligentní zpracování životopisů	Bezstavová	Interní
job_processor	Generování popisků pozic	Bezstavová	Interní
PostgreSQL (pgvector)	Transakční a auditní data	Stavová	Interní
SeaweedFS	Úložiště dokumentů	Stavová	Interní
RabbitMQ	Fronta zpráv pro asynchronní zpracování	Stavová	Interní
Umami	Webová analytika	Stavová (PostgreSQL)	Interní

Z hlediska provozního návrhu je významné zejména rozlišení mezi veřejně dostupnými službami, interními aplikačními komponentami a stavovými infrastrukturními službami. Veřejně dostupná část systému plní roli vstupního bodu pro externí uživatele, zatímco interní služby zajišťují zpracování aplikační logiky, integraci s okolními systémy a práci s daty. Stavové služby, jako jsou databáze, objektové úložiště nebo fronta zpráv, představují kritickou infrastrukturu, která nemá být přímo vystavena mimo vymezený provozní prostor.

Síťová segmentace proto vychází z principu omezení přímé dosažitelnosti komponent pouze na nezbytné komunikační vztahy. V praxi to znamená, že veřejná vrstva komunikuje s interní aplikační vrstvou, zatímco přístup ke stavovým službám je omezen na komponenty, které je ke své činnosti skutečně potřebují. Tento přístup podporuje princip nejmenších oprávnění a obranu v hloubce, tedy návrh, ve kterém bezpečnost systému není založena na jediném ochranném mechanismu, ale na kombinaci více vzájemně se doplňujících vrstev. Pokud by došlo k narušení jedné části systému, ostatní vrstvy stále omezují rozsah možného dopadu.

V kontextu této práce je síťová segmentace chápána především jako logický model oddělení provozních zón. Její konkrétní prosazení na úrovni firewallových pravidel, reverzní proxy, VLAN, směrování nebo dalších prvků síťové infrastruktury přesahuje rozsah aplikačního návrhu a spadá do kompetence oddělení odpovědného za provoz infrastruktury organizace KZ. Podstatné však je, že navržený model nasazení s tímto oddělením počítá a nevystavuje všechny komponenty systému stejnému bezpečnostnímu perimetru.

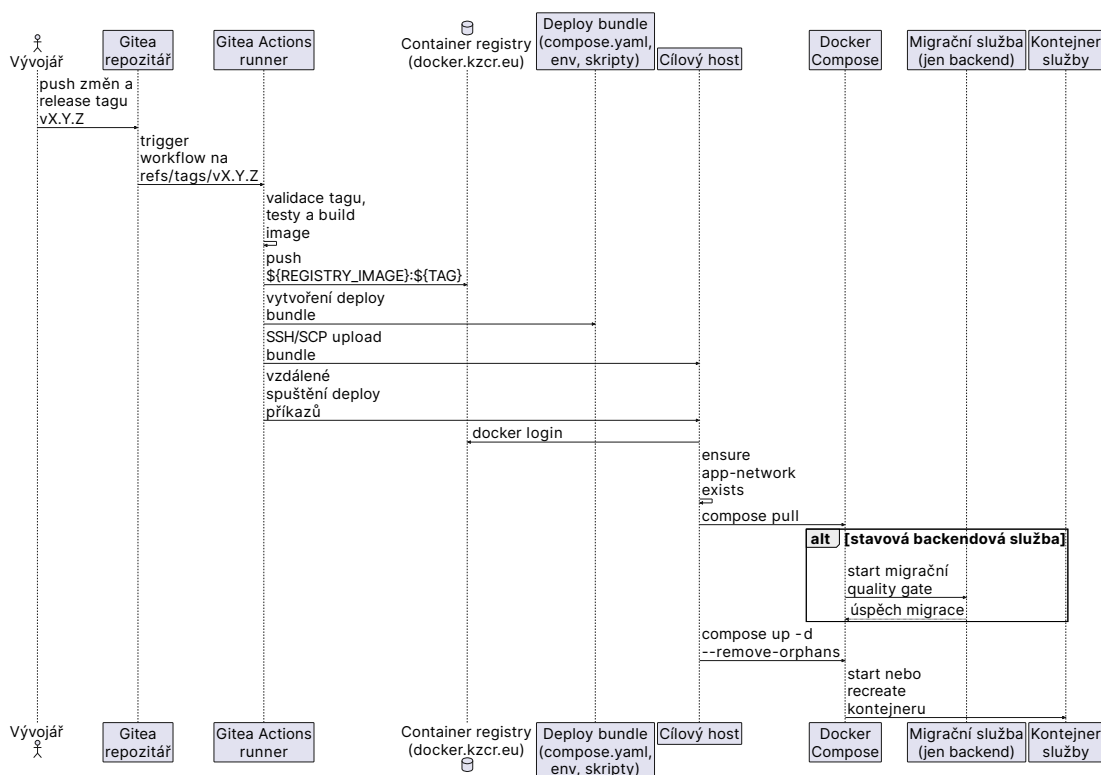
6.2 VYDÁNÍ A NASAZENÍ

Nasazení celého ekosystému služeb stavím na jednotném toku vydání založeném na obrazech. Nová verze vzniká v návaznosti na vydaný tag, prochází sestavením a ověřením a teprve poté je publikována do interního registru. Na cílový host se tak nepřenáší zdrojový kód k novému sestavení, ale již vytvořený obraz určený k provozu.

Z hlediska řízení změn je tento postup podstatný tím, že omezuje rozdíly mezi prostředím sestavení a prostředím nasazení. V organizaci s konzervativnějším změnovým režimem je výhodnější pracovat s již ověřeným obrazem než s variantou vzniklou až na cílovém serveru. Stejný princip zároveň zlepšuje dohledatelnost toho, která verze byla kdy nasazena, a usnadňuje návrat k dříve ověřenému stavu.

Praktickou realizaci tohoto toku zajišťuje automatizace v Gitea Actions, která po vydání nové verze provede sestavení obrazu, jeho publikaci do registru a podle typu služby také předání minimálního nasazovacího balíčku na cílový host. Pro text práce je však podstatnější samotný princip reprodukovatelného vydání než konkrétní syntaxe použité pipeline.

Obrázek 6. 1 schematicky zachycuje tento tok vydání od vzniku nové verze přes její ověření a uložení do registru až po distribuci na cílový host.



Obrázek 6. 1: Obecný tok vydání založený na obrazech s distribucí do registru a nasazením na cílový host

U různých částí řešení se tento obecný princip uplatňuje s odlišnou mírou provozní vazby. Backend je svázán s migracemi schématu a s konzistencí sdílených dat. V provozním modelu proto běží kontejner migration jako povinná závislost před startem služby hr-backend a při neúspěšném dokončení migrace se backend nespustí. Jiné služby se liší spíše typem konfigurace nebo hardwarovými požadavky. Společným jmenovatelem však zůstává, že nasazení vychází z již vytvořeného obrazu, **nikoli z ručních zásahů** na cílovém serveru.

Významnou provozní vlastností je i možnost návratu ke starší verzi výběrem dříve publikovaného obrazu. Omezením zůstává závislost na interním registru a na disciplíně při verzování a uchování vydaných obrazů.

6.3 SPRÁVA KONFIGURACE A BEZPEČNOST PROVOZU

Konfigurace systému je v navrženém řešení pojata jednotně jako vrstva oddělená od aplikačního kódu a od samotného obrazu. Základním principem je rozlišení mezi nasazovaným obrazem, běžnou provozní konfigurací a citlivými údaji. Tento

přístup umožňuje použít shodný obraz ve více prostředích a jeho konkrétní chování přizpůsobit až při nasazení, aniž by bylo nutné zasahovat do zdrojového kódu nebo vytvářet odlišné varianty aplikace pro každé prostředí zvlášť.

V praktické rovině se tento princip projevuje prostřednictvím konfiguračních souborů a proměnných prostředí předávaných mimo verzovaný repozitář. U části řešení jsou vybrané netajné proměnné vloženy už ve fázi sestavení, kdy je automatizace v Gitea Actions předá procesu tvorby obrazu a jejich hodnoty se promítnou do výsledného obrazu. Typicky jde o parametry, které musí být známy již při sestavení klientské aplikace. Naproti tomu provozní a citlivé údaje nejsou do obrazu zapisovány, ale jsou připojeny až při nasazení, a to buď prostřednictvím hostitelské konfigurace, nebo pomocí minimálního nasazovacího balíčku obsahujícího soubor `compose.yaml` a odpovídající konfigurační proměnné pro konkrétní verzi.

U služeb typu `hiring_backend`, `cv_processor` i `onboarding.kzcr.eu` tak zůstávají citlivé hodnoty, jako jsou přístupové údaje k databázi, integrační klíče nebo certifikáty, odděleny od obrazu aplikace. Přínosem tohoto modelu je omezení rizika nechtěného zveřejnění citlivých údajů při distribuci systému. Nevýhodou naopak zůstává závislost na disciplinovaném provozním postupu a na správném předání konfigurace do cílového prostředí.

Z hlediska dalšího rozvoje by bylo vhodné tento princip posílit zavedením centralizované samostatně provozované správy citlivých údajů, která by odpovídala požadavkům on-premise prostředí KZ. Takové řešení by omezilo závislost na lokálních souborech a dílčích předávacích mechanismech, zlepšilo dohledatelnost přístupů k tajným hodnotám a podpořilo jednotnější správu konfigurace v rámci celého ekosystému služeb.

6.4 DOHLEDOVÁ VRSTVA

Dohledová vrstva slouží k průběžnému sledování stavu nasazeného systému. V on-premise prostředí nelze spoléhat na dohled poskytovaný cloudovou platformou, a proto musí být součástí řešení vlastní mechanismy pro sběr, ukládání a vyhodnocování provozních dat. Ta pomáhají včas rozpoznat výpadek služby, zhoršení odezvy, chybnou konfiguraci nebo zaseknuté asynchronní zpracování.

Použité nástroje shrnuje Tabulka 6. 2. Pro tuto práci je však důležitější jejich společná funkce než jednotlivé produktové detaily. Centralizace logů, metrik a upozornění zkracuje dobu mezi vznikem incidentu a jeho pochopením, což je zvlášť důležité tam, kde se provozní problém může projevit až zprostředkovaně v personálním procesu.

Tabulka 6. 2: Role komponent dohledové vrstvy

Komponenta	Primární funkce	Provozní přínos
Promtail	Sběr a předzpracování logů z kontejnerů	Centralizovaná diagnostika bez zásahu do business logiky
Loki	Uložení a dotazování log streamů	Rychlá analýza incidentů podle času a štítků
Prometheus	Sběr metrik a trendové vyhodnocení	Podpora řízení dostupnosti a výkonu
Grafana	Vizualizace a upozornění	Jednotné operátorské rozhraní pro logy i metriky

Dohledová vrstva v této pilotní fázi projektu pokrývá především upozornění nad metrikami, které mají přímý vztah k dostupnosti systému a k průchodnosti hlavních procesů. Sleduje se zejména dostupnost endpointu `hr-backend/ready` a kontrolních endpointů dalších interních služeb z Tabulka 6. 1, HTTP metriky veřejných i interních rozhraní, především odezva a podíl odpovědí s kódy 5xx, stáří nejstarší čekající a právě zpracovávané položky v outboxu, počet položek přecházejících do chybového stavu, připravenost konzumentů fronty zpráv a základní kapacitní ukazatele stavových služeb, zejména databáze a objektového úložiště. Do stejné skupiny patří i upozornění na degradaci nebo zpomalování služeb a na neobvyklý nárůst interních chyb, pokud se opakovaně váží ke konkrétní integrační vazbě nebo provozní komponentě.

Z provozního hlediska je tento výběr důležitý tím, že nezachycuje pouze úplný výpadek služby, ale i stav postupné degradace. Ta se v personálním procesu často projeví nepřímo, například opožděným rozesíláním notifikací, zastavením asynchronního zpracování, selháním integrační vazby nebo prodlužováním odezvy při práci s dokumenty. Rozsah dohledové vrstvy je proto pro pilotní nasazení zvolen tak, aby pokrýval jak technickou dostupnost jednotlivých komponent, tak i provozní kontinuitu klíčových procesů, na nichž je navržené řešení závislé.

6.5 PROVOZNÍ OMEZENÍ A MITIGACE

Navržený model nasazení není univerzálně nejlepší variantou, ale pragmatickým kompromisem mezi provozní jednoduchostí, reprodukovatelností a oddělením složitějších částí systému. Oproti jediné nasazovací jednotce přináší více obrazů, více závislostí a vyšší nároky na koordinaci změn. Oproti distribuovanému prostředí řízenému plnohodnotným orchestrátorem však zachovává nižší infrastrukturní náročnost.

Nejvýraznější omezení plyne ze zvolené orchestrace pomocí Docker Compose, která je vhodná pro pilotní a menší produkční rozsah, ale nenabízí vlastnosti běžné u robustnějších orchestrátorů, například automatické rozložení zátěže mezi více uzly nebo samoozdravné mechanismy napříč hosty. Provoz je současně závislý na interním registru obrazů, správném vedení provozní konfigurace a na lokálních administrativních postupech organizace.

Tato omezení jsou v současné fázi částečně kompenzována využitím reprodukovatelného toku vydání, oddělením konfigurace, síťovou segmentací a zavedením dohledové vrstvy. Navržený přístup je proto plně dostačující pro pilotní podmínky KZ a představuje stabilní mezikrok.

Vzhledem k plné kontejnerizaci aplikací je však systém architektonicky připraven na budoucí rozvoj. Logickým krokem pro překonání limitů nástroje Docker Compose je migrace na plnohodnotnou orchestraci pomocí Kubernetes. Tento přechod nativně řeší výpadky během aktualizací zavedením plynulého nasazování (rolling updates) a umožňuje dosažení vysoké dostupnosti (HA) včetně dynamického škálování bezstavových komponent napříč více uzly. Zavedení Kubernetes by zároveň poskytlo standardizované prostředí pro nasazení plnohodnotné API Gateway a integraci nástroje pro centralizovanou správu citlivých údajů (např. HashiCorp Vault).

7 UŽIVATELSKÉ TESTOVÁNÍ A ZPĚTNÁ VAZBA

Uživatelské ověření jsem zařadil do práce proto, že u personálního systému nestačí pouze technická správnost implementace. Pokud uživatel nerozumí stavu náboru, neví, kdo má provést další krok, nebo se v rozhraní ztrácí, procesní přínos systému se rychle vytrácí. Cílem této kapitoly proto není statisticky reprezentativní výzkum celé organizace, ale pilotní ověření toho, zda navržené řešení podporuje klíčové role v jejich každodenních scénářích a kde je ještě potřeba systém zpřesnit.

7.1 CÍL A RÁMEC PILOTNÍHO OVĚŘENÍ

Cílem pilotního uživatelského ověření bylo zodpovědět tři praktické otázky. První z nich se týkala průchodnosti hlavních scénářů bez nadměrné asistence. Druhá mířila na místa, kde se uživatelé zastavují, vracejí nebo si nejsou jisti významem dalšího kroku. Třetí sledovala, zda lze získanou zpětnou vazbu převést do konkrétních doporučení pro stabilizaci a další rozvoj řešení.

Na základě těchto cílů jsem formuloval následující výzkumné otázky:

1. Dokážou cílové role dokončit klíčové scénáře bez externí asistence?
2. Ve kterých krocích vzniká nejčastěji nejistota, zdržení nebo chybné rozhodnutí?
3. Které prvky rozhraní uživatelům pomáhají udržet kontext procesu a které jej naopak komplikují?
4. Jaká zjištění mají nejvyšší prioritu pro další iteraci systému?

7.2 METODICKÝ POSTUP A VÝBĚR RESPONDENTŮ

Pilotní uživatelské ověření jsem vedl jako moderované scénářové ověření. Každý účastník procházel scénáře odpovídající jeho roli, pracoval s daty připomínajícími reálný provoz a průběžně komentoval, jak rozumí jednotlivým krokům a stavům systému. Tento postup mi umožnil nesledovat pouze to, zda uživatel úkol dokončil, ale také proč se v určitém bodě zastavil a jak interpretoval význam zobrazených informací.

Z metodického hlediska jsem kombinoval tři vzájemně se doplňující techniky. Základem bylo scénářové ověření s jednoznačně definovaným cílovým stavem. Druhou vrstvou tvořilo průběžné komentování postupu uživatelem, které pomohlo odlišit pouhou chybu od hlubší nejasnosti v terminologii nebo navigaci. Třetí vrstvou byl krátký rozhovor po dokončení scénáře, ve kterém jsem ověřoval, co bylo pro uživatele přínosné, co považoval za problematické a jaké změny by očekával před širším provozním rozšířením.

Respondenty jsem vybíral účelově podle rolí, které reprezentují tři rozdílné pohledy na tentýž proces. Nešlo tedy o snahu pokrýt všechny varianty uživatelů v organizaci, ale ověřit, zda je systém srozumitelný z perspektivy operativní personální práce, manažerského dohledu a nástupu nového zaměstnance.

Tabulka 7. 1: Zastoupení rolí v pilotním uživatelském ověření

Role	Účel zapojení	Hlavní testovaná agenda
HR specialista	Operativní ověření náborového workflow	Správa uchazečů, změny stavů, plánování pohovorů, komunikace s kandidáty
Vedoucí pracovník	Ověření rozhodovacích a dohledových kroků	Hodnocení kandidátů, přehled stavu náboru, vstupní agenda nového zaměstnance
Nový zaměstnanec	Ověření srozumitelnosti onboardingového rozhraní	Plnění onboardingových kroků, práce s dokumenty, notifikace a orientace v úkolech

7.3 TESTOVACÍ SCÉNÁŘE A HODNOTICÍ KRITÉRIA

Testovací scénáře jsem navrhl tak, aby pokryly kritické průchody systémem od založení pracovní pozice až po onboarding nového zaměstnance. Každý scénář měl jasně definovaný cílový stav, protože právě ten rozhoduje o tom, zda uživatel úlohu skutečně dokončil, nebo pouze rozhraním prošel bez jistoty, co provedl.

Tabulka 7. 2: Sada testovacích scénářů

ID	Scénář	Primární role	Vazba na požadavky
S1	Založení a publikace nové pracovní pozice	HR	R2, R3
S2	Zpracování přihlášky a změna stavu uchazeče	HR	R3, R6
S3	Naplánování pohovoru a odeslání pozvánek	HR	R3
S4	Převod uchazeče na zaměstnance a spuštění onboardingu	HR	R4
S5	Vyplnění onboardingového kroku a nahrání požadovaných dokumentů	Zaměstnanec	R4
S6	Kontrola reportu náborové průchodnosti	Vedoucí	R6
S7	Ověření notifikace a reakce na přidělený úkol	HR / Zaměstnanec	R4, R6

Scénáře v Tabulka 7. 2 záměrně nesledují jen izolované klikací úlohy, ale uzly, ve kterých se rozhoduje o průchodnosti celého procesu. Pokud se uživatel ztratí při změně stavu uchazeče, při převodu na zaměstnance nebo při práci s onboardingovým úkolem, nevzniká pouze lokální nepohodlí v rozhraní, ale přímé zpomalení navazujících kroků.

Protože šlo o pilotní uživatelské ověření kvalitativního charakteru, nesoustředil jsem se na jedinou agregovanou metriku. Vhodnější bylo sledovat kombinaci ukazatelů, které společně vystihují, zda je proces pro uživatele čitelný, dokončitelný a provozně použitelný. Zajímalo mě zejména to, zda uživatel dosáhl cílového stavu, zda potřeboval zásah moderátora, kolik chybných kroků udělal a jak svůj postup následně slovně hodnotil.

Tabulka 7. 3: Ukazatele použité v pilotním uživatelském ověření

Metrika	Typ	Interpretace
Dokončení scénáře	Kvantitativní	Zda uživatel dosáhl cílového stavu bez zásahu do dat nebo návratu na začátek
Potřeba asistence	Kvantitativní	Počet situací, kdy bylo nutné vysvětlit význam stavu, kroku nebo navigace
Doba dokončení	Kvantitativní	Orientační čas potřebný k dosažení cílového stavu u jednotlivých scénářů
Chybové kroky a návraty	Kvantitativní	Počet chybných voleb, zbytečných návratů nebo slepých míst v rozhraní
Komentáře uživatelů	Kvalitativní	Slovní popis přínosů, nejistot a navrhovaných úprav
Závažnost zjištění	Expertní	Priorita opravy podle dopadu na průchod procesem a četnosti opakování

7.4 HLAVNÍ ZJIŠTĚNÍ A IMPLIKACE PRO DALŠÍ ROZVOJ

Získaná pozorování ukázala, že největší hodnotu uživatelé vnímali v centralizaci informací, ve sjednocení stavu kandidátů a v lepší dohledatelnosti odpovědností napříč náborovým a onboardingovým procesem.

Jako problematická se naopak ukázala místa, kde systém předpokládá vyšší procesní znalost uživatele, než jakou lze očekávat při prvním použití. Typicky šlo o potřebu jasněji vysvětlit význam stavů, lépe zvýraznit další očekávaný krok a oddělit informace, které jsou pouze informativní, od těch, které vyžadují akci. U onboardingových scénářů byla důležitá také viditelná vazba mezi úkolem, termínem a odpovědnou rolí. Bez ní se uživatel sice v systému orientuje, ale hůře chápe prioritu jednotlivých kroků.

Tabulka 7. 4: Šablona souhrnných výsledků pilotního uživatelského ověření

Scénář	Dokončení scénáře	Potřeba asistence	Medián času	Počet chyb	Priorita zjištění
S1	doplnit	doplnit	doplnit	doplnit	doplnit
S2	doplnit	doplnit	doplnit	doplnit	doplnit
S3	doplnit	doplnit	doplnit	doplnit	doplnit
S4	doplnit	doplnit	doplnit	doplnit	doplnit
S5	doplnit	doplnit	doplnit	doplnit	doplnit
S6	doplnit	doplnit	doplnit	doplnit	doplnit
S7	doplnit	doplnit	doplnit	doplnit	doplnit

Z hlediska dalšího rozvoje z toho plyne poměrně jednoznačný závěr. Nejbližší iterace nemají směřovat především k rozšiřování funkcí, ale k úpravám, které zpřesní terminologii, zvýrazní odpovědnost za další krok a lépe propojí úkol, termín a roli. Právě v těchto bodech se rozhoduje, zda systém skutečně zrychlí nábor a adaptaci, nebo zda pouze digitalizuje stávající nejistotu. Získaná zjištění proto v další kapitole přímo převádím do doporučení pro další směřování vývoje.

8 NÁVRH DALŠÍHO SMĚŘOVÁNÍ VÝVOJE

Tato kapitola navazuje na výsledky analýzy, architektonického návrhu, implementace, nasazení i pilotního uživatelského ověření systému. Jejím cílem není navrhnout co největší množství nových funkcí, ale určit takové pořadí kroků, které udrží procesní přínos řešení pro KZ, sníží provozní rizika a současně ponechá prostor pro další rozvoj bez zbytečného architektonického dluhu.

Navrhovaný rozvoj stavím na třech principech. Prvním je preference změn s přímým dopadem na každodenní práci HR pracovníků, vedoucích a nových zaměstnanců. Druhým je zachování architektonické kázně, aby rychlé provozní úpravy neoslabily modularitu systému. Třetím principem je měřitelnost. Každá další iterace má být vyhodnocena nejen podle toho, že byla nasazena, ale i podle toho, zda zlepšila průchodnost procesu, srozumitelnost rozhraní a provozní stabilitu.

8.1 PRIORITIZAČNÍ RÁMEC ROZVOJE

Pro potřeby plánování jsem další vývoj rozdělil do tří horizontů. Do krátkodobého, střednědobého a dlouhodobého. Toto členění mi umožňuje oddělit kroky, které je vhodné provést ihned po stabilizaci pilotního provozu, od změn, jež vyžadují širší integrační nebo organizační připravenost.

Prioritizace vychází z vazby na požadavky R1-R6 a NF01-NF12. Nejvyšší prioritou mají oblasti, které současně zvyšují provozní spolehlivost, bezpečnost a srozumitelnost práce se systémem. Pilotní uživatelské ověření navíc ukázalo, že další vývoj nemá směřovat pouze k novým funkcím, ale i k lepšímu vysvětlení stavů, odpovědností a návazností mezi kroky procesu.

8.2 KRÁTKODOBÁ FÁZE (0 - 6 MĚSÍCŮ)

Krátkodobý horizont by měl být zaměřen především na stabilizaci provozu a na odstranění nejednoznačností, které se ukázaly při pilotním ověření. První prioritou je proto sjednocení monitorování a provozního vyhodnocování napříč aplikační vrstvou a vrstvou inteligentního zpracování dat. Cílem není sbírat více technických dat samoúčelně, ale získat spolehlivý obraz o tom, kde vznikají latence, selhání nebo nedoručené asynchronní operace.

Druhou prioritou je zpřesnění uživatelského rozhraní v bodech, kde testování ukázalo zvýšenou míru nejistoty. Prakticky to znamená lépe vysvětlit význam stavů kandidáta, viditelněji zobrazit odpovědnost za další krok vstupní agendy a seskupit onboardingové úkoly podle procesu, nikoli pouze podle technického typu záznamu. Tyto úpravy nejsou architektonicky rozsáhlé, ale mají okamžitý dopad na reálné používání systému.

Třetí oblastí krátkodobého rozvoje je bezpečnostní a provozní hygiena prostředí. Patří sem pravidelná rotace přístupových údajů, zpřesnění práce s tajnými hodnotami, kontrola obnovitelnosti záloh a formalizace postupů při nedostupnosti vrstvy inteligentního zpracování dat. Jde o změny, které nejsou z pohledu uživatele nejviditelnější, ale mají zásadní význam pro důvěryhodnost a dlouhodobou provozuschopnost systému.

8.3 STŘEDNĚDOBÁ FÁZE (6 - 18 MĚSÍCŮ)

Ve střednědobém horizontu má smysl soustředit se na rozšíření funkcí, které zvyšují řídicí hodnotu systému. První oblastí je rozvoj reportingu a procesní analytiky. Vedení organizace potřebuje sledovat nejen počet otevřených pozic, ale i průchodnost náboru, dobu reakce mezi kroky, úspěšnost adaptačních plánů a rozdíly mezi odštěpnými závody.

Druhou oblast představuje další rozvoj integračních vazeb systému, zejména ve vztahu k externím registrům a navazujícím interním agendám (Vema, Plánování služeb, atd.). Rozvoj by měl zůstat řízený přes explicitní integrační kontrakty, aby nedocházelo k tomu, že externí systémy začnou určovat podobu doménového modelu.

Třetím směrem střednědobého rozvoje je vyšší automatizace vstupní agendy a adaptace. Přínos zde nepřinese jen více notifikací, ale hlavně lepší práce s pravidly, eskalacemi a přehledem nesplněných kroků. Cílem je omezit situace, kdy je úkol sice v systému založen, ale z pohledu uživatele není zřejmé, kdo jej má převzít a co blokuje další postup.

8.4 DLOUHODOBÁ FÁZE (18+ MĚSÍCŮ)

Dlouhodobý rozvoj by měl být veden jako řízená evoluce již zavedené hybridní architektury. Nejde tedy o přechod k hybridnímu modelu, protože ten je součástí řešení už nyní, ale o rozhodování, zda mají být některé další části systému postupně oddělovány podle skutečných provozních dat. Takový krok dává smysl pouze tehdy, pokud konkrétní komponenta dlouhodobě vykazuje odlišný rytmus změn, zátěžový profil nebo provozní požadavky než zbytek jádra.

Vedle architektonické evoluce bude v delším horizontu důležitější i datové governance. S růstem objemu historických dat poroste potřeba jednotně definovat metriky, popsat význam reportovaných ukazatelů a určit odpovědnost za jejich interpretaci. Bez této vrstvy by se i technicky kvalitní systém postupně měnil v další zdroj nejednoznačných dat.

Samostatnou dlouhodobou oblastí je také kapacitní a kvalitativní rozvoj vrstvy inteligentního zpracování dat. Pokud se její výstupy stanou běžnou součástí náboru a adaptace, bude nutné systematicky vyhodnocovat přesnost, latenci, provozní náklady i dopady případných chyb. Rozvoj této vrstvy proto nesmí být veden jen technologickou atraktivitou, ale jasně měřeným přínosem pro proces.

8.5 ŘÍZENÍ ZMĚNY A ORGANIZAČNÍ PŘEDPOKLADY

Úspěch dalšího rozvoje nebude záviset jen na technických rozhodnutích, ale i na tom, zda organizace udrží pravidelný cyklus vyhodnocování změn. Do tohoto cyklu musí vstupovat vývoj, provoz i vlastníci HR procesů. Bez společného vyhodnocení hrozí, že technické priority začnou sledovat interní pohodlí týmu místo reálných problémů uživatelů.

Stejně důležitá je průběžná práce se zpětnou vazbou. Pilotní uživatelské ověření ukázalo, že i drobná nejasnost v názvu stavu nebo ve zobrazení odpovědnosti může zpomalit celý proces více než absence nové funkce. Změny proto doporučuji nasazovat inkrementálně, ověřovat je na konkrétních scénářích a teprve poté rozšiřovat do širšího provozu.

8.6 ZÁVĚREČNÉ DOPORUČENÍ KAPITOLY

Pro další směřování vývoje považuji za nejvhodnější evoluční postup. Nejprve je potřeba stabilizovat provoz a odstranit bariéry zjištěné při uživatelském ověření, poté rozšiřovat analytiku, integrace a automatizaci a teprve nakonec otevírat otázku hlubších strukturálních změn. Takto zvolený postup dává nejvyšší šanci, že systém bude dlouhodobě podporovat cíle digitalizace náboru a adaptace bez zbytečného provozního rizika.

Navržený plán současně zachovává prostor pro budoucí organizační i technické změny v KZ, aniž by narušil základní architektonické principy stanovené v této práci.

9 ZÁVĚR

Tato diplomová práce vycházela z praktického problému, který se v prostředí Krajské zdravotní, a.s. dlouhodobě projevoval jako roztržitá správa náboru a adaptace pracovníků. Analytická část ukázala, že hlavním omezením není jediný nefunkční krok, ale souběh několika faktorů, mezi které patří rozptýlená data, ruční komunikace mezi rolemi, nízká transparentnost stavu náboru, chybějící auditní stopa a papírově vedená adaptace nových zaměstnanců. V organizaci s více odštěpnými závody a vysokou frekvencí nástupů se tyto slabiny navzájem zesilují a postupně snižují propustnost celého procesu.

Na základě zjištěného stavu jsem formuloval požadavky na cílové řešení a porovnal je s dostupnými komerčními produkty. Rešerše ukázala, že na trhu existují kvalitní dílčí nástroje pro správu náboru nebo onboarding, ale žádné z hodnocených řešení současně nenaplnuje požadavek na procesní kontinuitu, provoz na vlastní infrastrukturu, multi-tenantní členění organizace a vazbu na specifika českého zdravotnického prostředí. Tato zjištění vedla k rozhodnutí navrhnout a dále rozvíjet vlastní řešení, které dokáže uvedené požadavky spojit v jednom datovém a procesním modelu.

V návrhové části byla představena hybridní architektura kombinující modulární monolit pro transakční jádro a oddělenou vrstvu pro inteligentní zpracování dat. Tento přístup umožňuje zachovat provozní jednoduchost systému a zároveň podporuje jeho rozšiřitelnost, auditovatelnost a integrační schopnosti. Součástí práce je také implementace backendu, kariérního portálu, onboardingového rozhraní, integračních adaptérů a nasazení v on-premise prostředí.

Uživatelské ověření potvrdilo, že hlavním přínosem řešení je sjednocení informací, zvýšení transparentnosti procesu a omezení ruční komunikace mezi zúčastněnými rolemi. Zároveň se ukázalo, že pro úspěch systému je klíčová nejen jeho technická kvalita, ale i srozumitelnost procesů, jasné rozdělení odpovědností a průběžná práce se zpětnou vazbou uživatelů.

Hlavním přínosem práce je propojení analytického, architektonického a implementačního pohledu na konkrétní problém organizace. Výsledkem není pouze návrh, ale prakticky využitelný základ platformy respektující specifika zdravotnického prostředí. Limitem práce zůstává nutnost dalšího rozvoje systému v návaznosti na provozní zkušenosti a dostupné kapacity.

Práce nicméně prokazuje, že vlastní softwarové řešení představuje v daném kontextu efektivnější alternativu k rigidním komerčním systémům. Zároveň poskytuje metodický rámec pro realizaci obdobných digitálních transformací ve zdravotnických organizacích.

POUŽITÁ LITERATURA

- [1] ŘEPA, Václav. *Podnikové procesy: procesní řízení a modelování*. 2., aktualiz. a rozš. vyd. Praha: Grada, 2007. ISBN 978-80-247-2252-8. 281 s.
- [2] MINISTERSTVO ZDRAVOTNICTVÍ ČR. *Jak zajistit dostatek zdravotnického personálu? Česká republika a WHO hledají řešení pro budoucnost*. Online. 7. 2025. Dostupné z: <https://mzd.gov.cz/tiskove-centrum-mz/jak-zajistit-dostatek-zdravotnickeho-personalu-ceska-republika-a-who-hledaji-reseni-pro-budoucnost/>. [citováno 2026-02-08].
- [3] TEAMIO. *Automatic import of vacancies*. Online. 2026. Dostupné z: <https://www.teamio.com/en/what-it-can-do/automatic-import-of-vacancies/>. [citováno 2026-02-27].
- [4] TEAMIO. *Pricing*. Online. 2026. Dostupné z: <https://sk.teamio.com/en/pricing/>. [citováno 2026-02-27].
- [5] TEAMIO. *Vyskusat zadarmo*. Online. 2026. Dostupné z: <https://sk.teamio.com/vyskusat-zadarmo/>. [citováno 2026-02-27].
- [6] RECRUITIS.IO. *ATS Recruitis: Naborovy software*. Online. 2026. Dostupné z: <https://www.recruitis.io/>. [citováno 2026-02-27].
- [7] DATACRUIT. *Modern recruitment software with AI*. Online. 2026. Dostupné z: <https://www.datacruit.com/>. [citováno 2026-02-27].
- [8] DATACRUIT. *Price list of the Datacruit ATS system*. Online. 2026. Dostupné z: <https://www.datacruit.com/en-gb/pricing>. [citováno 2026-02-27].
- [9] SLONEEK. *Recruitment Platform (ATS) - Candidate Management*. Online. 2026. Dostupné z: <https://www.sloneek.com/ats/>. [citováno 2026-02-27].
- [10] ONBEE.APP. *Onboarding and Offboarding; digitalni karta zamestnan-ce, HRIS*. Online. 2026. Dostupné z: <https://onbee.app/cz/>. [citováno 2026-02-27].
- [11] SAP. *What is SAP SuccessFactors?*. Online. 2026. Dostupné z: <https://www.sap.com/products/hcm/what-is-sap-successfactors.html>. [citováno 2026-02-27].
- [12] SAP. *SAP SuccessFactors Onboarding*. Online. 2026. Dostupné z: <https://www.sap.com/products/hcm/employee-onboarding.html>. [citováno 2026-02-27].

- [13] WORKDAY. *Unified Solutions*. Online. 2026. Dostupné z: <https://www.workday.com/en-us/products/human-capital-management/unified-solutions.html>. [citováno 2026-02-27].
- [14] WORKDAY. *Talent Acquisition and Recruiting Software*. Online. 2026. Dostupné z: <https://www.workday.com/en-us/products/talent-management/talent-acquisition.html>. [citováno 2026-02-27].
- [15] ORACLE. *Human Capital Management (HCM)*. Online. 2026. Dostupné z: <https://www.oracle.com/human-capital-management/>. [citováno 2026-02-27].
- [16] ORACLE. *Oracle Recruiting and Recruiting Booster*. Online. 2026. Dostupné z: <https://www.oracle.com/human-capital-management/recruiting/>. [citováno 2026-02-27].
- [17] PORTAL GOV.CZ. *Overeni udaju o zdravotnickem pracovníkovi*. Online. 2026. Dostupné z: <https://portal.gov.cz/sluzby-vs/vypis-udaju-o-zdravotnickem-pracovnikovi-S1348>. [citováno 2026-02-27].
- [18] BASS, Len; Paul CLEMENTS a Rick KAZMAN. *Software Architecture in Practice*. 2. Addison-Wesley, 2003. ISBN 978-0-321-15495-8.
- [19] VERNON, Vaughn. *Implementing Domain-Driven Design*. Addison-Wesley, 2013. ISBN 978-0-321-83457-7.
- [20] COCKBURN, Alistair. *Hexagonal architecture*. Online. 2005. Dostupné z: <https://alistair.cockburn.us/hexagonal-architecture/>. [citováno 2026-03-17].
- [21] NEWMAN, Sam. *Building Microservices: Designing Fine-Grained Systems*. 2. O'Reilly Media, 2019. ISBN 978-1-492-03402-5.
- [22] EVANS, Eric. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003. ISBN 978-0-321-12521-7.
- [23] EVANS, Eric. *DDD Reference: Definitions and Pattern Summaries*. Online. 2015. Dostupné z: https://www.domainlanguage.com/wp-content/uploads/2016/05/DDD_Reference_2015-03.pdf. [citováno 2026-04-07].